Computers & Operations Research 160 (2023) 106407

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

Exact approaches for the unconstrained two-dimensional cutting problem with defects

Hao Zhang[1], Shaowen Yao[1], Qiang Liu, Jiewu Leng, Lijun Wei *

Key Laboratory of Computer Integrated Manufacturing System, School of Electromechanical Engineering, Guangdong University of Technology, 510006, China

ARTICLE INFO

Keywords:
Packing
Cutting
Defect
Integer programming
Dynamic programming

ABSTRACT

This paper studies the unconstrained two-dimensional cutting problem with defects, which requires cutting a set of rectangular item types from a rectangular sheet with defects. The available number of each type is not limited. Two versions of the problem are studied, based on whether the guillotine cut constraint is required. Due to equipment limitations, the guillotine cutting restrictions are essential in specific industries, requiring cutting from one side to the other with each cut and splitting the sheet into two separate sheets. An integer model is proposed for the non-guillotine cut version, and an exact dynamic programming (DP) approach is proposed for the guillotine cut version. To reduce the number of vertical and horizontal positions, we extend the normal patterns, raster points, and meet-in-the-middle patterns to consider the effect of defects. The experimental results on standard benchmarks show that the proposed model can solve most of the instances in literature in a short time. For the guillotine cut version, our approach can significantly reduce the number of cut positions and improve the efficiency of the DP approach compared to the previous exact approach that considers all possible positions. Our approach solves the large instances with more than 300 item types in a relatively short time.

1. Introduction

Cutting problems are often required in many industries, including metal, wooden, glass, paper, and leather (Wei et al., 2019). Thousand of papers have been proposed since the seminal paper of Gilmore and Gomory (1961). However, only a few papers address the cutting problem with defects in the literature as witnessed by Iori et al. (2021), although this problem has recently attracted increasing interest from academic circles. The presence of defects is often encountered in many manufacturing industries. For example, in the furniture industry, wooden boards may contain knotholes. In the glass industry, the glass may contain defects inherent to the float process (Parreño et al., 2020). Furthermore, the presence of defects may change the property of the standard cutting problem. For example, using the widely used normalized patterns (Herz, 1972) directly to solve the cutting problem with defects may miss the optimal solution (Afsharian et al., 2014). Therefore, there is still a large research gap for the cutting problem with defects. It is required to investigate the effect of defects and improve the efficiency of existing models or approaches for the cutting problem with defects.

This paper studies the unconstrained two-dimensional cutting problem with defects. The objective is to cut a subset of items among given types from a given sheet so that the total profit of selected items is maximized. The term "unconstrained" means that the number of available items is not limited for each type. The unconstrained case occurs in the cutting stock problem with large-scale production and weakly heterogeneous items (Birgin et al., 2012), although the constrained case (limited available number) is more often encountered in practice. Moreover, the unconstrained case is theoretically crucial as it can provide helpful profit upper bounds for the constrained case (Wei and Lim, 2015; Russo et al., 2020). This paper aims to study the effect of the defects on the cutting problem and extend existing well-used techniques, such as normal patterns, raster points, and meet-in-the-middle (MIM) patterns, to solve the cutting problem with defects.

Formally, in this problem, we are given a set of item types $I = \{1, 2, \ldots, n\}$, and a large rectangular sheet S_0 with width W_0 and height H_0. Each type $i \in I$ has width w_i^s, height h_i^s, and a non-negative profit value v_i. Assume the bottom-left corner of the input sheet is located at the origin point of the x and y coordinate axes (see Fig. 1). An additional set of rectangles $D = \{1, 2, \ldots, m\}$ is given to represent

* Corresponding author.
E-mail addresses: zhanghao0815@126.com (H. Zhang), yaoshaowen97@foxmail.com (S. Yao), liuqiang@gdut.edu.cn (Q. Liu), villagerwei@gmail.com (L. Wei).

[1] The first author and the second author contributed equally to this work.

https://doi.org/10.1016/j.cor.2023.106407
Received 24 March 2023; Received in revised form 24 August 2023; Accepted 25 August 2023
Available online 31 August 2023
0305-0548/© 2023 Elsevier Ltd. All rights reserved.

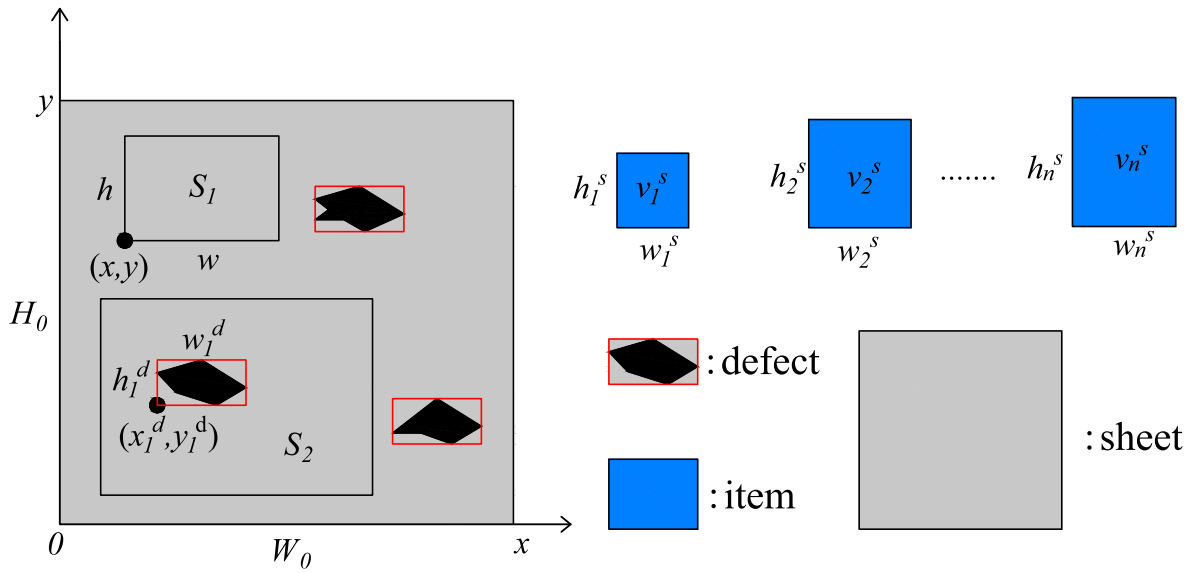


Fig. 1. Two-dimensional cutting problem with defects.

the defects. Note that rectangles are used to approximate the defect regions that may be irregular. Each defect $j \in D$ is associated with its bottom-left point (x_j^d, y_j^d) and its size $w_j^d \times h_j^d$. Without loss of generality, it is assumed that all the input positions and sizes are integers. The objective is to select and cut a set of items and maximize the cut items value. Moreover, a feasible cutting pattern must satisfy the following constraints:

- all the cut items must lie entirely within the sheet.
- there is no overlap among cut items.
- there is no overlap between cut items and defects.
- the orientation of each item type is fixed, i.e., the rotation of small items is not allowed.

The available number for each item type is unlimited. According to the typology of Wäscher et al. (2007), this problem is called the two-dimensional, rectangular, unconstrained, single large object placement problem with a large defective object (2D_U_SLOPP_D). Note that according to whether the value of each item is equal to its area or not, the problem can be classified as unweighted and weighted version (Fayard et al., 1998), respectively. We study both versions in this paper. The *guillotine-cut* constraint, which requires all the cuts must be of the guillotine type, i.e., the cut path must be parallel to the sides of the sheet to divide it into two completely separate sheets (Iori et al., 2021; Wei et al., 2014), is often required in the industry. Two versions of the problems, *non-guillotine* and *guillotine*, are studied in this paper. The guillotine version is also called the two-dimensional, rectangular, unconstrained, guillotineable-layout, single large object placement problem with a large defective object (2D_UG_SLOPP_D) in the literature.

There are three papers addressing the 2D_U_SLOPP_D. Gonçalves and Wäscher (2020) presented a MIP model and a biased random-key genetic heuristic algorithm, and Luo et al. (2022) proposed a biased genetic algorithm hybridized with variable neighborhood search. If the defect is not considered, we get the two-dimensional, rectangular, unconstrained, single-large object placement problem (2D_U_SLOPP), studied by several literature papers. To solve it, Birgin et al. (2012) proposed a recursive partitioning approach, which combines the recursive five-block heuristic (Morabito and Morales, 1998) and the L-approach (Lins et al., 2003). Later, de Queiroz et al. (2014) negatively answered the question raised by Birgin et al. (2012) whether the L-approach could find the optimal solution by finding an instance where it failed. Recently, Wei and Lim (2015) proposed a best-fit branch-and-bound heuristic and got the best results for this problem.

Regarding the guillotine version, the 2D_UG_SLOPP_D is first introduced by Carnieri et al. (1993), where only a single defect is considered. Dynamic programming (DP)-based heuristic, including a branch-and-bound search in which the bounds are calculated by assuming no defects, is presented. Vianna and Arenales (2006) introduced a branch-and-bound algorithm based on an AND/OR-graph approach and a heuristic search combining a depth-first search. They also proposed a few instances, including more than one defect. Later, Neidlein et al. (2009) introduced an improved bound based on the method by Vianna and Arenales (2006). Note that the discretization sets (normal patterns) proposed by Herz (1972) were used by Vianna and Arenales (2006) and Neidlein et al. (2009) to reduce the number of positions. However, as stated by Afsharian et al. (2014), both methods might fail to find the optimal solution because of structural and computational limitations. Afsharian et al. (2014) proposed a dynamic programming algorithm, which can find the optimal solution by considering all integer positions for vertical and horizontal guillotine cuts, and a DP-based heuristic approach, using a reduced set of positions, that found a high percentage of optimal solutions for some randomly generated instances. Subsequently, Yin et al. (2019) presented an improved heuristic dynamic programming algorithm to solve the multi-defect problem and to reduce further the discrete set of positions proposed by Afsharian et al. (2014). Recently, Martin et al. (2020) addressed the constrained version of this problem. A compact integer linear programming model, a Benders decomposition exact algorithm, and a constraint programming-based exact algorithm were proposed. Later, Zhang et al. (2023) proposed an exact approach based on a recursive dynamic programming approach.

There are also several papers considering other cutting problems with defects. Tuzun Aksu and Durak (2016) introduced a DP-based algorithm for an online glass-cutting problem with defects. The problem involves cutting a continuous ribbon of flat glass conveyed on a belt to obtain various-sized glass products, accounting for the location and size of various defects detected in real-time by a camera. Consequently, the cutting decisions necessitate swift online resolution, with each decision being made within seconds. Durak and Tuzun Aksu (2017) studied the online glass-cutting problem with defects and production targets. The online problem is transformed into a series of static cutting problems, and two methods (a DP algorithm and a mixed-integer programming-based algorithm) were proposed to solve the static problem. The 2018 ROADEF/EURO challenge glass cutting problem can be regarded as a special variant of the bin packing problem (Wei et al., 2013) with

defects, which requires packing a set of rectangular items into a minimum set of larger rectangular bins. This base problem has been studied by several papers. Libralesso and Fontan (2021) presented an anytime tree search algorithm, and Parreño et al. (2020) proposed a beam search algorithm. Parreño and Alvarez-Valdes (2021) proposed the first integer linear model for this problem and obtained the optimal solution for small instances. Martin et al. (2022) studied the two-stage and one-group two-dimensional guillotine cutting problems with defects and proposed a constraint programming-based algorithm and an integer linear programming formulation. Rao and Luo (2022) proposed a grey wolf optimization algorithm for the irregular packing problem with defects. Besides, Neidlein et al. (2016) introduced a problem generator for the two-dimensional rectangular single large object placement problem with defects.

If the defect is not considered, we can get the two-dimensional, rectangular, unconstrained, guillotineable-layout, single large object placement problem (2D_UG_SLOPP). Several exact approaches have been introduced to this problem in the literature. The widely used method is recursive DP based on a top-down approach. Most of them use the knapsack function proposed by Gilmore and Gomory (1966), where an exact basic DP and an improved version were introduced. Later, Herz (1972) suggested a recursive top-down algorithm and introduced the discretization points (also called normal patterns lately) to reduce the number of cuts. The cut positions are obtained by the conic combination of the dimensions of the items. Using DP and the one-dimensional knapsack to get the lower and upper bounds, Hifi and Zissimopoulos (1996) proposed an improved version of the exact recursive algorithm by Herz (1972). Afterward, Hifi (1997) proposed a hybrid algorithm that combines a tree-search procedure and a heuristic based on solving a series of one-dimensional knapsack problems by DP algorithms. Fayard et al. (1998) presented a heuristic algorithm that extends their best strips cutting algorithm. Russo et al. (2013) pointed out three errors in the original procedure by Gilmore and Gomory (1966) and proposed an improvement of the knapsack function-based algorithm. Later, based on a new DP algorithm modified from the one by Russo et al. (2013), Russo et al. (2014) presented five improvements and several computational refinements, getting the best results for this problem. Boschetti et al. (2016) investigate the application of GPU computing of the DP-based scheme by Beasley (1985a). Recently, Masone et al. (2022) further improve the algorithm from Russo et al. (2013).

Another exact approach for 2D_UG_SLOPP is the branch and bound based on the bottom-up approach. Young-Gun and Kang (2002) proposed a new upper bound based on one-dimensional relaxation and applied the new bound to the recursive algorithm by Hifi and Zissimopoulos (1996). This bound was later combined by Young-Gun et al. (2003) into a best-first branch and bound algorithm based upon the bottom-up approach. Kang and Yoon (2011) suggested an improved version of this algorithm by removing the dominated patterns efficiently and avoiding duplicated patterns.

The normal patterns and raster points are widely used to reduce the number of cut positions in the literature. As stated above, the normal patterns (called discretization points initially) were first introduced by Herz (1972) for the 2D_UG_SLOPP while the raster points are first introduced by Terno et al. (1987) to reduce normal patterns. Another concept equivalent to the normal patterns is the useful numbers proposed by Carnieri et al. (1994). Later, Fischer and Scheithauer (2015) and Scheithauer (2018) extended the normal patterns and raster points to consider the effect of the defects. Recently, Côté and Iori (2018) proposed the MIM patterns, which mix the concept of normal patterns and useful numbers, to reduce the number of patterns, and the extensive computational tests on several relevant C&P problems showed the efficiency of these techniques. A theoretical analysis of these different grids of points and a computational study in the 2D knapsack problem is proposed by de Almeida Cunha et al. (2020).

This paper introduces an integer model for the 2D_U_SLOPP_D and a DP-based approach for the 2D_UG_SLOPP_D. The normal patterns, raster points, and MIM patterns are utilized to reduce the points and speed up the approaches. There are three main contributions of this paper. Firstly, we extend the MIM patterns by considering defects. To the best of our knowledge, this is the first paper to do that for the cutting problem with defects. Secondly, an integer programming model is proposed for the 2D_U_SLOPP_D, and a DP-based approach is proposed for 2D_UG_SLOPP_D. Thirdly, we solved the integer programming model using the commercial solver CPLEX and conducted extensive experiments to test the effect of the normal patterns, raster points, and MIM patterns. The computational results on the standard benchmark show that our approach can reduce the time significantly compared to the exact approach considering all integer positions. Besides, the exact DP-based approach finds the optimal solution in a short time, even for large instances.

The paper is organized as follows. Firstly, some fundamental definitions are introduced in Section 2. After that, the normal patterns, raster points, and MIM patterns considering the defects are described in Section 3. Then, Section 4 introduces the integer model for the 2D_U_SLOPP_D and Section 5 presents the exact DP approach for 2D_UG_SLOPP_D. Section 6 gives the experimental tests on the standard benchmark. Finally, we summarize our conclusions in Section 7.

2. Fundamental definitions

Before introducing our approach, we first give some fundamental definitions.

2.1. Sub-sheet

A rectangular sub-sheet is part of the input sheet. Let the dimensions of a sub-sheet be (w, h) and let the coordinates of its bottom-left corner be (x, y) (see Fig. 1), then, a sub-sheet can be represented by $S = (x, y, w, h)$. The original input large sheet S_0 can be denoted as $(0, 0, W_0, H_0)$.

2.2. Contaminated and pure sheets

Definition 1 (*Contaminated Sheet*)**.** A sub-sheet $S = (x, y, w, h)$ is called contaminated if it contains or overlaps with at least one defect.

Definition 2 (*Pure Sheet*)**.** A sub-sheet $S = (x, y, w, h)$ is called pure if it does not contain or overlap with any defects.

Let the 0–1 function $c(x, y, w, h)$ denote whether the sheet $S = (x, y, w, h)$ is a contaminated sheet or not, i.e., $c(x, y, w, h)$ is equal to 1 if (x, y, w, h) is a contaminated sheet. The function $c(x, y, w, h)$ can be calculated by the following equation:

$$c(x, y, w, h) = \begin{cases} 0, & \text{if } x \geq x_j^d + w_j^d \text{ or } y \geq y_j^d + h_j^d \text{ or } x_j^d \\ & \geq x + w \text{ or } y_j^d \geq y + h, \forall j \in D \\ 1, & otherwise \end{cases}.$$

In Fig. 1, S_1 is a pure sheet while S_2 is a contaminated sheet.

3. Grids of points considering defects

In the next sections, we use the following examples to support the presentation of the three discrete points concepts of normal patterns, raster points, and MIM patterns.

Example 1. $W_0 = H_0 = 27$, four ($n = 4$) items with size $5 \times 5, 10 \times 10, 12 \times 12$, and 15×15, and a single ($m = 1$) defect located at $(9, 9)$ with size 2×2. The value of each item is equal to its area.

Example 2. $W_0 = H_0 = 30$, five ($n = 5$) items with size $6 \times 6, 9 \times 9, 12 \times 12, 13 \times 13$, and 16×16, and a single ($m = 1$) defect located at $(9, 9)$ with size 2×2. The value of each item is equal to its area.

3.1. Normal patterns, raster points, and MIM patterns for pure sheets

A packing pattern is called *normalized* if all items are adjusted as far left and down as possible without overlapping. Note that according to the definition, the most bottom-left item (the position of its bottom-left corner) in this pattern must be placed at the position $(0,0)$. Christofides and Whitlock (1977) showed that any two-dimensional cutting pattern can be replaced by an equivalent normalized pattern. Therefore, the set of positions for placing an item can be reduced by only considering the possible points in the normalized patterns (Herz, 1972). These points are called normal patterns. More exactly, the vertical and horizontal (here, the words vertical and horizontal refer to cutting direction) normal patterns on the large sheet can be taken from the sets $N^v(W_0)$ and $N^h(H_0)$, respectively, where the sets $N^v(w)$ and $N^h(h)$ for a generic sub-sheet with size (w,h) are defined as

$$N^v(w) = \left\{ z \mid z = \sum_{i\in I} \alpha_i w_i^s, 0 \le z \le w, \alpha_i \in \{0,1,2,\dots\} \right\},$$
$$N^h(h) = \left\{ z \mid z = \sum_{i\in I} \alpha_i h_i^s, 0 \le z \le h, \alpha_i \in \{0,1,2,\dots\} \right\}.$$

In other words, each element of the vertical (resp., horizontal) normal patterns is a non-negative integer linear combination of the widths (resp., heights) of the item types. We use a standard dynamic programming (DP) technique to calculate the discrete points, similar to the idea of dynamic programming for the one-dimensional knapsack problem (1KP). According to the definition, for Example 1, $N^v(W_0) = \{0,5,10,12,15,17,20,22,24,25,27\}$. Similarly, for Example 2, $N^v(W_0) = \{0,6,9,12,13,15,16,18,19,21,22,24,25,26,27,28,29,30\}$. Since the sheet, the items, and the defects are square in the above example, we will only consider the vertical sets even in the following for the example calculations.

Terno et al. (1987) and Scheithauer and Terno (1996) attempted to reduce normal patterns by using the raster points. Let $\overline{N^v(w)}$ (resp., $\overline{N^h(h)}$) represent the maximal value in $N^v(w)$ (resp., $N^h(h)$) and they are defined as

$$\overline{N^v(w)} = \max\left\{ z \mid z \in N^v(w) \right\},$$
$$\overline{N^h(h)} = \max\left\{ z \mid z \in N^h(h) \right\}.$$

For a given vertical position $z \in N^v(w)$, if $z+\overline{N^v(w-z)} < w$, then an item placement in position z leads to a loss of at least $w-z-\overline{N^v(w-z)}$ width units. If in $N^v(w)$ there is another position $z' > z$, such that $z'+\overline{N^v(w-z)} \le w$, then, placing any item at position z can be skipped, as an equivalent or better solution can be obtained by placing the item at position z', hence the best is to use $z' = \overline{N^v(w-\overline{N^v(w-z)})}$. Terno et al. (1987) defined the so-called vertical and horizontal raster points as

$$R^v(w) = \left\{ \overline{N^v(w-z)} \mid z \in N^v(w) \right\},$$
$$R^h(h) = \left\{ \overline{N^h(h-z)} \mid z \in N^h(h) \right\},$$

where, for example for $R^v(w)$, z plays the role of $\overline{N^v(w-z)}$ in the expression above for the best z'. It is easy to check that considering all z belonging to $N^v(w)$ does not add any value to $R^v(w)$ that cannot be obtained as the best z' mentioned above.

For Example 1, we have $R^v(W_0) = \{0,5,10,12,15,17,22,27\}$, where the values 20, 24 and 25 are removed compared to $N^v(W_0)$. While no item fits if placed in 24 or 25, when placing an item at position 20, the remaining width is 7, which can only be filled by the item 5×5. An equivalent solution can be obtained by placing the item at position 22 with remaining width of 5. Similarly, for Example 2, we have $R^v(W_0) = \{0,6,9,12,13,15,16,18,21,24,30\}$, with seven values removed compared to $N^v(W_0)$, including 19, 22, 25, 26, 27, 28, 29.

Côté and Iori (2018) proposed the MIM patterns. The basic idea is that for a given integer parameter $t(0 \le t \le w)$, all the items with the left corner placed at the left of t are pushed left along the width direction until they touch another item or left side of the sheet. On the other hand, all the remaining items (fully at the right of t) are pushed right until they touch another item or the sheet's right side. In other words, the items at the left of t are packed at a normal pattern while the remaining items are placed at a useful number (similar to the normal pattern while the items are pushed as far right and up as possible). Formally, for a given t, the vertical and horizontal MIM patterns for a given item $i \in I$ are defined as:

$$M^v(i,t,w) = M_l^v(i,t,w) \cup M_r^v(i,t,w),$$
$$M^h(i,t,h) = M_l^h(i,t,w) \cup M_r^h(i,t,w),$$

where

$$M_l^v(i,t,w) = \left\{ z \mid z = \sum_{k=1}^{n} \alpha_k w_k^s, 0 \le z \le \min(t-1, w-w_i^s), \alpha_k \in \{0,1,2,\dots\} \right\},$$
$$M_r^v(i,t,w) = \left\{ z \mid z = w - z' - w_i^s, z' = \sum_{k=1}^{n} \alpha_k w_k^s, 0 \le z' \le w - w_i^s - t, \alpha_k \in \{0,1,2,\dots\} \right\},$$
$$M_l^h(i,t,h) = \left\{ z \mid z = \sum_{k=1}^{n} \alpha_k h_k^s, 0 \le z \le \min(t-1, h-h_i^s), \alpha_k \in \{0,1,2,\dots\} \right\},$$
$$M_r^h(i,t,h) = \left\{ z \mid z = h - z' - h_i^s, z' = \sum_{k=1}^{n} \alpha_k h_k^s, 0 \le z' \le h - h_i^s - t, \alpha_k \in \{0,1,2,\dots\} \right\}.$$

Note that in the original definition, i is excluded while calculating the MIM patterns for item i with $\alpha_k = 1, k \ne i$. However, since the available number of each item is not limited (unconstrained) in our problem, the item i must be included, and any value must be allowed for each α_k. Based on the above definition, the MIM patterns for a given t are defined as:

$$M^v(t,w) = \bigcup_{i\in I} M^v(i,t,w),$$
$$M^h(t,h) = \bigcup_{i\in I} M^h(i,t,h).$$

The MIM patterns are the patterns with minimum size among all t values:

$$M^v(w) = \left\{ M^v(t^*,w) \mid t^* = \underset{t\in\{0,1,2,\dots,w\}}{\arg\min} \{ |M^v(t,w)| \} \right\},$$
$$M^h(h) = \left\{ M^h(t^*,h) \mid t^* = \underset{t\in\{0,1,2,\dots,h\}}{\arg\min} \{ |M^h(t,h)| \} \right\}.$$

For Example 1, if t is set to 8, we have $M_l^v(1,8,W_0) = \{0,5\}$, $M_r^v(1,8,W_0) = \{10,12,17,22\}$ and $M^v(1,8,W_0) = \{0,5,10,12,17,22\}$. Similarly, $M^v(2,8,W_0) = \{0,5,12,17\}$, $M^v(3,8,W_0) = \{0,5,10,15\}$, and $M^v(4,8,W_0) = \{0,5,12\}$. Finally, $M^v(8,W_0) = \{0,5,10,12,15,17,22\}$. Actually, $t = 8$ is the optimal value in this case. Therefore, $M^v(W_0) = M^v(8,W_0) = \{0,5,10,12,15,17,22\}$. Furthermore, the element 27 is removed in MIM pattern $M^v(W_0)$ compared to the raster points $R^v(W_0)$. This is because while calculating $M_r^v(i,t,w)$, we add the constraint to ensure that the resulting point is not larger than $w - w_i^s$. On the contrary, no such constraint is added while calculating the raster points. Similarly, for Example 2, we have $t^* = 12$, $M^v(W_0) = M^v(12,W_0) = \{0,6,9,12,14,15,17,18,21,24\}$, having again one element (i.e., 30) less than $R^v(W_0)$. The horizontal MIM patterns are calculated similarly.

3.2. Normal patterns, raster points, and MIM patterns for contaminated sheets

Considering a contaminated sub-sheet (x, y, w, h), define $D(x, y, w, h)$ as the set of defects overlapping with (x, y, w, h), formally,

$$D(x, y, w, h) = \left\{ j \mid x < x_j^d + w_j^d \text{ and } y < y_j^d + h_j^d \text{ and } x_j^d < x + w \text{ and } y_j^d < y + h, j \in D \right\}.$$

If $D(x, y, w, h) = \emptyset$, then the normal patterns of the vertical (resp., horizontal) cuts on the sheet (x, y, w, h) can be calculated by $N^v(w)$ (resp., $N^v(h)$).

Now considering the case $D(x, y, w, h) \neq \emptyset$, we define a feasible cutting pattern *normalized* if no item can be moved leftwards or downwards without overlapping with defects. For an item p packed into the cutting pattern, we use (x_p^s, y_p^s) to represent its bottom-left point and $i(q)$ to represent its type index. Therefore, the x_p^s value can be classified into three cases:

(a) 0: item p is the most left-packed item;
(b) $x_p^s = x_j^d + w_j^d$: p is right adjacent to defect j;
(c) $x_p^s = x_q^s + w_{i(q)}^s$: p is right adjacent to another packed item q. Hence, a sequence of adjacent items may result placed starting from the left sheet border (from case (a)) or from the right border of a defect (from case (b)).

Based on the above observation, we can induce the extended vertical and horizontal normal patterns to be used for the contaminated sheet (x, y, w, h) in order to cover any possible feasible normalized cutting pattern. Formally, for a sub-sheet $S = (x, y, w, h)$ that can be the result of other cuts, $N^v(x, y, w, h)$ and $N^h(x, y, w, h)$ define the relative coordinates of the possible cuts as follows, which will later support the definition of the extended raster points.

$$N^v(x, y, w, h) = N^v(w) \cup \left\{ z \mid z = x_j^d - x + w_j^d + v, j \in D(x, y, w, h), v \in N^v(w), 0 \le z \le w \right\},$$

$$N^h(x, y, w, h) = N^h(h) \cup \left\{ z \mid z = y_j^d - y + h_j^d + v, j \in D(x, y, w, h), v \in N^h(h), 0 \le z \le h \right\}.$$

Each element of the vertical (resp., horizontal) set is a combination of the widths of the items (resp., heights) plus either 0 or the value of the right (resp., top) point of any defect in the coordinate system relative to (x, y). The first part covers the case (a) and the sub-cases of (b) starting from 0 (left sheet border). The second part covers the case (c) and the sub-cases of (b) starting from any $x_j^d + w_j^d$ (right defect border). This proves the completeness of the proposed extended normal patterns. Similarly, we define $\overline{N^v(x, y, w, h)}$ ($\overline{N^h(x, y, w, h)}$) as the maximal value in the set $N^v(x, y, w, h)$ (resp., $N^h(x, y, w, h)$). For Example 1, we have $N^v(0, 0, W_0, H_0) = \{0, 5, 10, 11, 12, 15, 16, 17, 20, 21, 22, 23, 24, 25, 26, 27\}$, that is, with the additional values $\{11, 16, 21, 23, 26\}$ compared to $N^v(W_0)$, coming from the translation by 11 of the values $\{0, 5, 10, 12, 15\}$. Similarly, for Example 2, we have $N^v(0, 0, W_0, H_0) = \{0, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30\}$.

For two sets A and B, we define $A \oplus B$ as the set of elements consisting of the sum of any couple of elements, one from A and one from B, i.e.,

$$A \oplus B = \left\{ c \mid c = a + b, a \in A, b \in B \right\}.$$

Therefore, $N^v(x, y, w, h)$ and $N^h(x, y, w, h)$ can be rewritten as

$$N^v(x, y, w, h) = \left\{ z \mid z \in (RP(x, y, w, h) \oplus N^v(w)), 0 \le z \le w \right\},$$

$$N^h(x, y, w, h) = \left\{ z \mid z \in (TP(x, y, w, h) \oplus N^h(h)), 0 \le z \le h \right\}.$$

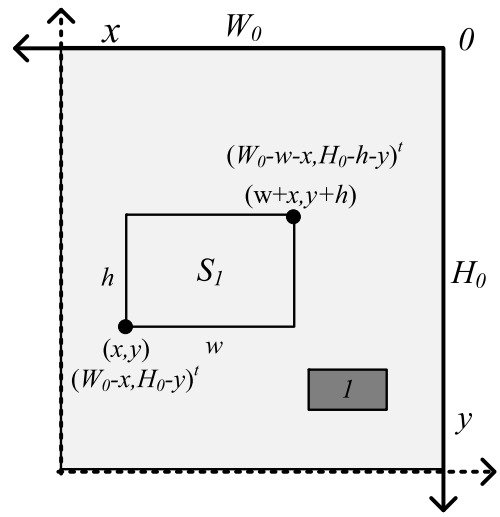


Fig. 2. New coordinate system used to calculate the raster points.

where

$$RP(x, y, w, h) = \{0\} \cup \left\{ z \mid z = x_j^d + w_j^d - x, j \in D(x, y, w, h) \right\},$$

$$TP(x, y, w, h) = \{0\} \cup \left\{ z \mid z = y_j^d + h_j^d - y, j \in D(x, y, w, h) \right\}.$$

$RP(x, y, w, h)$ includes 0 and the set of x values of the right point of each defect, and $TP(x, y, w, h)$ includes 0 and the set of y values of the top point of each defect.

To calculate the raster points for a contaminated sheet, we define the reversed coordinate system where the top-right corner of the input sheet represents the original point, and the direction of the x-axis and y-axis is reversed. Fig. 2 shows the reversed coordinate system. Given a point (x, y) in the first coordinate system, we use $(x', y')^t$ (where $x' = W_0 - x, y' = H_0 - y$) to represent the point in the new system. The sub-sheet (x, y, w, h) is represented as $(W_0 - x - w, H_0 - y - h, w, h)^t$ in the new relative coordinate system. We define $N^v(x', y', w, h)^t$ (resp., $N^h(x', y', w, h)^t$) as the vertical (resp., horizontal) normal patterns in the new relative coordinate system. The calculation of $N^v(x', y', w, h)^t$ and $N^h(x', y', w, h)^t$ is the same as the one of $N^v(x, y, w, h)$ and $N^h(x, y, w, h)$, similarly reversed inside the sub-sheet. In other words, it is obtained by rightward and upward moves from the perspective of the original coordinate system. Note that, if there are no defects on the sub-sheet, then $N^v(W_0 - x - w, H_0 - y - h, w, h)^t$ is the same as the set gotten by reversed coordinates from $N^v(x, y, w, h)$.

Similarly to the raster points for the pure sub-sheet, we define the extended vertical and horizontal raster points for a contaminated sub-sheet as follows.

$$R^v(x, y, w, h) = \left\{ \overline{N^v(x, y, w - z, h)} \mid z \in N^v(W_0 - x - w, H_0 - y - h, w, h)^t \right\},$$

$$R^h(x, y, w, h) = \left\{ \overline{N^h(x, y, w, h - z)} \mid z \in N^h(W_0 - x - w, H_0 - y - h, w, h)^t \right\}.$$

According to the definition, any normal coordinate z which belongs to $N^v(x, y, w, h)$ but not to $R^v(x, y, w, h)$ is absorbed by a z' with $z < z'$, and can be replaced by it. Note that the maximum element of $N^v(x, y, w, h)$ belongs for sure even to $R^v(x, y, w, h)$, considering the case z = 0. This means that z' is the maximum of a set $N^v(x, y, w - z'^*, h)$ where z'^* belongs to $N^v(W_0 - x - w, H_0 - y - h, w, h)^t$, and no other

maximum like that exists in the range $[z, \dots, z' - 1]$. What has to be assured to avoid missing possible feasible solutions, is that the right shift toward z' of any item packed within a feasible cutting pattern at the coordinate z, does not hit any defect. However, if $z' + w_i^s > x_j^d - x$ would hold for any item type i and defect j (while $z + w_i^s \le x_j^d - x$), then it would mean $z \le x_j^d - x - w_i^s < z'$. Hence, by selecting $z^* = w - x_j^d + x + w_i^s$ that belongs to $N^v(W_0 - x - w, H_0 - y - h, w, h)^t$, the maximum value of $N^v(x, y, w - z^*, h)$ falls in the range $[z, \dots, x_j^d - w_i^s - x]$ hence in the range $[z, \dots, z' - 1]$ which is a contradiction. More generally, the right shift of a sequence of adjacent items must similarly not hit any defect, but this is assured again by the same considerations where the sum of the widths of the involved items may replace w_i^s. This concludes the proof of completeness for the extended raster points.

For the Example 1, we have $N^v(0, 0, W_0, H_0)^t = \{0, 5, 10, 12, 15, 17, 18, 20, 22, 23, 24, 25, 27\}$, which, this time, compared to $N^v(W_0)$ contains the additional values $\{18, 23\}$ coming from the translation of 18 (in turn coming from $W_0 - x_1^d$) applied to the values $\{0, 5\}$. Therefore $R^v(0, 0, W_0, H_0) = \{0, 5, 10, 12, 15, 17, 22, 27\}$, which is the same as $R^v(W_0)$. Compared to the set $N^v(0, 0, W_0, H_0)$, the elements 11, 16, 20, 21, 23, 24, 25, and 26 are removed. Similarly, for Example 2, we have $N^v(0, 0, W_0, H_0)^t = \{0, 6, 9, 12, 13, 15, 16, 18, 19, 21, 22, 24, 25, 26, 27, 28, 29, 30\}$, resulting in the determination of $R^v(0, 0, W_0, H_0) = \{0, 6, 9, 11, 12, 13, 15, 17, 18, 21, 24, 30\}$.

The MIM patterns for a contaminated sub-sheet can be defined similarly. For a given parameter t, all the items with left corners at the left of t are pushed left along the width direction until another item blocks them, the left side of the sheet, or the right side of a defect. All the remaining items are pushed right along the width direction until another item, the sheet's right side, or the defect's left side blocks them. Therefore, for a sub-sheet (x, y, w, h), a given t, and a given item $i \in I$, the extended vertical and horizontal MIM patterns are defined as:

$$M^v(i, t, x, y, w, h) = M_l^v(i, t, x, y, w, h) \cup M_r^v(i, t, x, y, w, h),$$
$$M^h(i, t, x, y, w, h) = M_l^h(i, t, x, y, w, h) \cup M_r^h(i, t, x, y, w, h),$$

where

$$M_l^v(i, t, x, y, w, h) = \left\{ z \mid z \in N^v(x, y, w, h), 0 \le z \le \min(t - 1, w - w_i^s) \right\},$$
$$M_r^v(i, t, x, y, w, h) = \left\{ z \mid z = w - w_i^s - z', z' \in N^v(W_0 - x - w, H_0 - y - h, w, h)^t, \right.$$
$$\left. 0 \le z' \le w - w_i^s - t \right\},$$
$$M_l^h(i, t, x, y, w, h) = \left\{ z \mid z \in N^h(x, y, w, h), 0 \le z \le \min(t - 1, h - h_i^s) \right\},$$
$$M_r^h(i, t, x, y, w, h) = \left\{ z \mid z = h - h_i^s - z', z' \in N^h(W_0 - x - w, H_0 - y - h, w, h)^t, \right.$$
$$\left. 0 \le z' \le h - h_i^s - t \right\}.$$

The MIM patterns for a given t are defined as:

$$M^v(t, x, y, w, h) = \bigcup_{i \in I} M^v(i, t, x, y, w, h),$$
$$M^h(t, x, y, w, h) = \bigcup_{i \in I} M^h(i, t, x, y, w, h).$$

The MIM patterns for the sheet (x, y, w, h) are defined as:

$$M^v(x, y, w, h) = \left\{ M^v(t^*, x, y, w, h) \mid t^* = \underset{t \in \{0,1,\dots,w\}}{\arg\min} \{ |M^v(t, x, y, w, h)| \} \right\},$$
$$M^h(x, y, w, h) = \left\{ M^h(t^*, x, y, w, h) \mid t^* = \underset{t \in \{0,1,\dots,h\}}{\arg\min} \{ |M^h(t, x, y, w, h)| \} \right\}.$$

The completeness of the extended MIM patterns comes straightforward from the one of the extended normal patterns at the same way as happens for classic MIM patterns $M^v(w)$ and $M^h(h)$ with respect to classic normal patterns $N^v(w)$ and $N^h(h)$.

For the Example 1, $t^* = 8$ and $M^v(0, 0, W_0, H_0) = \{0, 5, 10, 12, 15, 17, 22\}$, which is the same as the $M^v(W_0)$. Similarly, for Example 2, $t^* = 9$ and $M^v(0, 0, W_0, H_0) = \{0, 6, 9, 11, 12, 14, 15, 17, 18, 21, 24\}$, with the element 11 added with respect to $M^v(W_0)$.

As the results in Examples 1 and 2 show, our expanded patterns for discrete points considering defects tend to enlarge the classical patterns at the minimum. Note that the classical approach using the standard normal patterns for Example 2 would fail to find the optimal solution. Furthermore, the proposed normal patterns and raster points for contaminated sheets are the same as those by Fischer and Scheithauer (2015) and Scheithauer (2018) although they are defined in different formulations.

4. Integer programming model for the 2D_U_SLOPP_D

In the following, for a given sheet (x, y, w, h), we use $X(x, y, w, h)$ and $Y(x, y, w, h)$ to represent the discrete points (normal patterns or raster points or MIM patterns) along the x-axis and y-axis, respectively. If the sheet (x, y, w, h) is pure, we just use $X(w)$ and $Y(h)$ for simplicity. Moreover, we use X_0 and Y_0 to represent the sets $X(0, 0, W_0, H_0)$ and $Y(0, 0, W_0, H_0)$, respectively. The binary decision variable z_{ixy} is equal to 1 if the bottom left corner of an item with type i is placed at the point (x, y); otherwise, it is zero. Based on the model of Beasley (1985b), the 2D_U_SLOPP_D can be modeled as follows:

$$\text{IP}: \quad \max \quad \sum_{i \in I} \sum_{x \in X_0} \sum_{y \in Y_0} v_i z_{ixy} \tag{1}$$

$$s.t \quad \sum_{i \in I} \sum_{\{x \in X_0 \mid p - w_i^s + 1 \le x \le p\}} \sum_{\{y \in Y_0 \mid q - h_i^s + 1 \le y \le q\}} z_{ixy} \le 1, \quad p \in X_0, q \in Y_0 \tag{2}$$

$$\sum_{i \in I} \sum_{\{x \in X_0 \mid x_j^d - w_i^s + 1 \le x \le x_j^d + w_j^d - 1\}} \sum_{\{y \in Y_0 \mid y_j^d - h_i^s + 1 \le y \le y_j^d + h_j^d - 1\}} z_{ixy} = 0, \quad j \in D \tag{3}$$

$$z_{ixy} \in \{0, 1\}, \quad i \in I, x \in X_0, y \in Y_0 \tag{4}$$

where constraint (2) means that each point (p, q) should only be covered once and (3) means that each defective point should not be covered by any item.

5. DP-based approach for the 2D_UG_SLOPP_D

For a sub-sheet $S = (x, y, w, h)$, let $F(x, y, w, h)$ be the maximal value we can get from this sheet using guillotine cut. If this sheet is pure, the value of $F(x, y, w, h)$ depends on the size of the sheet (w, h), and we write $F(x, y, w, h)$ as $F(w, h)$ for simplicity.

For a pure sheet with size (w, h), we define $g(w, h)$ as the maximal value we can get by guillotine cutting only one type of item. Since the orientation of each item is fixed, $g(w, h)$ can be calculated by the following equation.

$$g(w, h) = \max\left\{ 0, \left\{ v_i \times \left\lfloor \frac{w}{w_i^s} \right\rfloor \times \left\lfloor \frac{h}{h_i^s} \right\rfloor \;\middle|\; i = 1, \dots, n \right\} \right\},$$

where $\lfloor a \rfloor$ is the largest integer number less than or equal to a. We use the following recursive function proposed by Beasley (1985a) to compute the optimal value for the general guillotine cutting of the sheet (w, h):

$$F(w, h) = \max \left\{ \begin{array}{l} g(w, h), \\ \max\left\{ 0, \left\{ F(z, h) + F(w - z, h) \mid z \le w/2, z \in X(w) \backslash \{0\} \right\} \right\}, \\ \max\left\{ 0, \left\{ F(w, z) + F(w, h - z) \mid z \le h/2, z \in Y(h) \backslash \{0\} \right\} \right\} \end{array} \right\}.$$

Note that $F(w, h)$ is equal to 0 if $w < w_{\min}$ or $h < h_{\min}$, where $w_{\min} = \min\{w_i^s \mid i \in I\}$ and $h_{\min} = \min\{h_i^s \mid i \in I\}$ are items minimum width and height, respectively. Additionally, due to symmetry, only half of the widths (resp., heights) are considered when considering the sub-sheet.

For a contaminated sub-sheet (x, y, w, h), all the possible vertical and horizontal cuts in the set $X(x, y, w, h)$ and $Y(x, y, w, h)$ must be considered. This is because there is generally no symmetry property after considering the defects. Therefore, the following recursive equation can calculate $F(x, y, w, h)$.

$$F(x, y, w, h) = \begin{cases} F(w, h), & \text{if } c(x, y, w, h) = 0 \\ \max \begin{Bmatrix} 0, \\ F(x, y, z, h) + F(x + z, y, w - z, h) \mid z \in X(x, y, w, h) \backslash \{0\}, \\ F(x, y, w, z) + F(x, y + z, w, h - z) \mid z \in Y(x, y, w, h) \backslash \{0\} \end{Bmatrix}, & \text{otherwise} \end{cases}.$$

The F is calculated using a top-down approach, which is the same as the approach of Beasley (1985a).

6. Computational results

All our DP approaches were implemented as a sequential algorithm in Java (JDK 7 updated 21, 64-bit edition), and no multi-threading was explicitly used. All experiments described in this section were conducted on a personal computer equipped with an Intel Core(TM) i7-7700 CPU clocked at 3.60 GHz (GHz) with 16 GB (GB) of RAM, running a Windows 10 (64-bit) operating system. Besides, the tool for solving the integer programming models is ILOG CPLEX 12.7.1 (64-bit edition).

6.1. Test sets

There are two sets in the literature for cutting problems with defects.

The first set includes 14 weighted instances. The first eight instances with a single defect are introduced by Carnieri et al. (1993). The size of the input sheet and the item types is identical among these instances. The only difference is the position and size of the defect. The width and height of the input sheet are set to 200 and 100, respectively. Besides, the number of item types is set to 5. Vianna and Arenales (2006) expanded this test set by providing six additional instances with more than one defect. The large sheet and the item types are identical to the one of Carnieri et al. (1993). Details of this test problem set can also be found in Neidlein et al. (2016).

The second set contains unweighted instances and is generated by Afsharian et al. (2014) by adapting the problem generator of Neidlein et al. (2016). These instances can be classified into three categories based on the size of the large sheet. The large sheet area is set to 5,625, 22,500, and 90,000 square units for categories 1, 2, and 3, respectively. Besides, two different shapes of the large sheet are considered for each category: (a) quadratic (75, 75) and non-quadratic (112, 50) for category 1, (b) quadratic (150, 150) and non-quadratic (225, 100) for category 2, (c) quadratic (300, 300) and non-quadratic (450, 200) for category 3. Note that the sheet area in the non-quadratic (112, 50) type of category 1 is slightly less than 5,625. The items and defects are generated as follows.

- five different values, 5, 10, 15, 20, and 25, are considered for the number of item types n.
- the width and height of the items are generated uniformly in the interval $[W_0/p^s, 3W_0/4]$ and $[H_0/p^s, 3H_0/4]$, respectively, where p^s is the parameter used to control the start point of the interval. Three different values, 6, 8, and 10, are considered for p^s.
- four different values, 1, 2, 3, and 4, are considered for the number of defects m.
- the width and height of the defects are generated uniformly in the interval $[W_0/10, W_0/6]$ and $[H_0/10, H_0/6]$, respectively. The coordinates of each defect j are generated uniformly in the intervals $[0, W_0 - w_j^d]$ and $[0, H_0 - h_j^d]$. For simplicity, overlapping is avoided in the generation of defects.

By considering the combination of two different shapes for the input sheet, five different number of item types, and three different p^s settings for the item sizes, then 30 $(2 \times 5 \times 3)$ different classes can be gotten for each category. For each class and each value of m ($m \in \{1, 2, 3, 4\}$), 15 instances are generated randomly. Therefore, 5400 $(3 \times 30 \times 4 \times 15)$ instances were obtained, 60 in each class of each category. They may also be separated in 30 groups (3 categories, 2 shapes, 5 item types numbers) of three classes. Hence, each category contains 10 groups and each group contains 180 instances. The classes included in each group share the sheet size and the number of item types but have a different setting for the item size.

The third set of instances is modified from those introduced in the 2018 ROADEF/EURO challenge (Libralesso and Fontan, 2021) to test the limitations of our approach. Since multiple bins exist in the original instances, only the first bin is used as input sheet. The size of the original bin is 6000×3210, which makes our approaches run out of memory. Therefore, we divide the dimension of the bin and items by 20 and round it down to the nearest integer. The resulting size of the sheet is 300×160. The position and the size of the defects are handled in a similar way, and the size of each defect is set to one if the division results to zero. The sequential and staged constraints from (Libralesso and Fontan, 2021) are not considered. There are three subsets A, B, and X, and 50 instances in total. The number of defects is in the range $[0, 8]$.

Table 1
Number of discrete points for the instances in the first set.

#Inst.	n	m	$\lvert X_0^n \rvert$	$\lvert X_0^r \rvert$	$\lvert X_0^m \rvert$	$\lvert Y_0^n \rvert$	$\lvert Y_0^r \rvert$	$\lvert Y_0^m \rvert$
1	5	1	49	21	20	52	22	21
2	5	1	49	21	20	50	21	20
3	5	1	49	21	20	50	21	20
4	5	1	54	22	20	50	21	20
5	5	1	54	22	20	50	21	20
6	5	1	49	21	20	50	21	20
7	5	1	49	21	20	52	21	20
8	5	1	49	21	20	52	21	20
9	5	4	59	33	31	70	43	40
10	5	4	59	25	23	60	31	29
11	5	4	59	25	23	60	31	29
12	5	4	83	37	35	71	30	28
13	5	4	58	27	25	60	29	27
14	5	2	59	25	23	50	21	20
avg.			55.6	24.4	22.9	55.5	25.3	23.9

6.2. Number of points of different discrete sets

Table 1 lists the number of x-axis and y-axis discrete points for the instances in the first set. The columns $|X_0^n|$ (resp., $|Y_0^n|$), $|X_0^r|$ (resp., $|Y_0^r|$) and $|X_0^m|$ (resp., $|Y_0^m|$) are the number of normal patterns, raster points and MIM patterns on the x-axis (resp., y-axis). The row $avg.$ is the average result for set 1. The average number of discrete points on the x-axis for the MIM pattern is 22.9, with 23.9 on the y-axis, which is approximately 58% less than the normal patterns and slightly less than the raster points.

Tables 2 and 3 list the number of x-axis and y-axis discrete points for the instances in the second set. The columns $|X_0^n|$ (resp., $|Y_0^n|$), $|X_0^r|$ (resp., $|Y_0^r|$) and $|X_0^m|$ (resp., $|Y_0^m|$) are the average number of normal patterns, raster points and MIM patterns on the x-axis (resp., y-axis) for each partition of 45 instances with the given sheet and item types number, respectively. The rows $avg.$ is the average results for five partitions with given sheet size and number of defects, and the $avg.$ in the last row of the table indicates the total average for all partitions with the given number of defects. Compared to the normal patterns, the number of raster points and MIM patterns reduces significantly. Especially for the instances with a larger sheet ($300 * 300$ or $450 * 200$), the number of raster points and MIM patterns is only half with respect to normal patterns. Furthermore, the number of raster points is generally less than the number of MIM patterns, but the gap is

Table 2
Number of x-axis discrete points for the instances in the second set.

Size	n	Pure			$m=1$			$m=2$			$m=3$			$m=4$		
		$\lvert X_0^n\rvert$	$\lvert X_0^r\rvert$	$\lvert X_0^m\rvert$	$\lvert X_0^n\rvert$	$\lvert X_0^r\rvert$	$\lvert X_0^m\rvert$	$\lvert X_0^n\rvert$	$\lvert X_0^r\rvert$	$\lvert X_0^m\rvert$	$\lvert X_0^n\rvert$	$\lvert X_0^r\rvert$	$\lvert X_0^m\rvert$	$\lvert X_0^n\rvert$	$\lvert X_0^r\rvert$	$\lvert X_0^m\rvert$
75*75	5	11.5	6.7	7.6	12.7	7.3	7.9	13.7	8.0	8.4	15.2	8.6	8.9	16.4	9.4	9.8
	10	25.9	15.9	16.7	27.0	16.9	17.4	28.2	17.6	18.0	29.1	18.4	18.6	30.4	19.4	19.5
	15	30.9	19.1	20.0	32.0	20.1	20.6	33.2	21.2	21.5	33.8	21.8	21.9	34.7	22.8	22.7
	20	38.6	27.5	28.3	39.1	28.6	28.8	39.4	29.3	29.4	40.1	30.0	29.8	40.9	30.9	30.6
	25	40.9	31.6	32.0	41.2	32.4	32.3	41.8	33.2	33.0	42.3	34.0	33.6	43.0	34.8	34.3
	avg.	29.6	20.1	20.9	30.4	21.0	21.4	31.3	21.9	22.1	32.1	22.6	22.6	33.1	23.5	23.4
112*50	5	14.5	8.4	9.4	16.3	9.2	9.9	18.2	9.9	10.8	20.3	10.6	11.5	22.4	11.7	12.6
	10	37.2	20.4	22.0	38.9	21.5	22.8	41.6	23.0	24.1	43.3	24.1	25.2	44.4	25.5	26.2
	15	48.0	28.6	30.0	49.3	29.9	30.9	50.0	31.2	31.8	51.4	33.0	33.4	52.7	34.4	34.4
	20	55.4	37.1	38.2	56.1	38.3	38.8	57.3	39.7	39.9	58.0	40.6	40.5	58.8	41.6	41.3
	25	60.6	44.3	45.1	60.9	45.2	45.5	61.8	46.1	46.3	62.6	47.2	47.1	63.1	48.2	47.9
	avg.	43.1	27.8	29.0	44.3	28.8	29.6	45.8	30.0	30.6	47.1	31.1	31.5	48.3	32.3	32.5
150*150	5	11.1	6.7	7.2	12.2	7.1	7.5	13.9	7.6	8.2	15.3	8.3	8.9	16.9	9.0	9.5
	10	35.9	18.4	19.8	38.4	19.7	21.1	39.7	20.5	21.9	41.6	21.6	22.7	43.9	23.0	24.0
	15	42.8	20.8	23.0	44.0	21.7	23.5	45.6	23.3	24.6	47.3	24.4	25.5	48.9	25.7	26.6
	20	61.9	35.8	37.3	63.1	36.8	38.0	64.0	38.2	39.0	65.5	39.6	40.1	66.5	40.7	41.1
	25	72.0	46.8	48.2	72.6	48.3	48.9	73.5	49.4	49.7	74.3	50.5	50.6	75.4	51.6	51.6
	avg.	44.7	25.7	27.1	46.1	26.7	27.8	47.3	27.8	28.7	48.8	28.9	29.6	50.3	30.0	30.6
225*100	5	16.5	8.5	9.8	19.4	9.7	10.6	21.7	10.3	11.3	23.6	11.6	12.3	25.6	12.5	13.2
	10	46.4	19.0	22.5	49.9	20.2	23.4	52.7	21.8	24.4	54.4	23.2	25.5	57.2	24.9	27.1
	15	69.4	32.1	34.8	71.9	33.8	35.9	74.8	35.7	37.6	77.2	37.0	38.8	80.0	39.1	40.4
	20	93.5	51.0	53.4	95.3	52.8	54.6	97.3	54.9	56.3	99.2	57.5	58.5	101.7	59.4	60.1
	25	100.4	56.7	58.6	102.1	58.2	59.8	103.8	59.6	61.0	105.4	62.0	62.9	107.0	64.1	64.7
	avg.	65.2	33.5	35.8	67.7	34.9	36.8	70.1	36.5	38.1	72.0	38.2	39.6	74.3	40.0	41.1
300*300	5	13.8	7.1	8.3	15.8	7.7	8.6	18.0	8.6	9.3	19.8	9.5	10.3	22.5	10.3	11.3
	10	35.8	14.6	17.3	39.5	16.1	18.3	42.2	17.6	19.6	45.7	18.9	20.8	48.6	20.2	21.9
	15	62.9	25.3	28.2	66.0	26.4	29.0	70.2	27.7	30.4	72.0	29.0	31.3	75.0	30.8	33.1
	20	91.9	41.4	44.7	94.3	42.7	45.5	97.7	44.5	47.2	100.0	45.8	48.4	103.0	48.0	50.3
	25	111.8	51.9	56.0	113.9	54.0	57.5	115.8	56.0	59.2	117.3	57.9	60.7	119.3	59.8	62.3
	avg.	63.2	28.1	30.9	65.9	29.4	31.8	68.8	30.9	33.1	70.9	32.2	34.3	73.7	33.8	35.8
450*200	5	17.9	8.4	10.1	20.7	9.3	10.8	23.3	9.9	11.5	26.0	11.1	12.4	29.4	12.3	13.5
	10	55.1	20.5	24.1	59.2	21.7	25.1	64.7	23.4	26.4	67.6	25.1	28.1	70.6	26.1	29.2
	15	98.7	34.2	38.3	103.3	35.6	39.2	107.7	37.2	40.7	113.4	39.7	43.1	117.7	42.3	45.5
	20	124.9	46.7	51.5	128.6	48.3	52.7	131.8	50.0	54.1	137.1	52.6	56.7	142.1	55.6	59.4
	25	170.9	74.4	79.9	172.9	76.3	81.6	176.1	78.5	83.5	180.2	81.0	85.7	182.2	83.8	88.3
	avg.	93.5	36.8	40.8	97.0	38.2	41.9	100.7	39.8	43.3	104.9	41.9	45.2	108.4	44.0	47.2
avg.		56.6	28.7	30.8	58.6	29.9	31.6	60.7	31.1	32.6	62.6	32.5	33.8	64.7	33.9	35.1

insignificant. In other words, no dominance relationship exists between raster points and MIM patterns. In addition, the number of all discrete points increases with the number of defects and the sheet size.

6.3. Results for the 2D_U_SLOPP_D

The IP model (1)–(4) is used to solve the instances of the non-guillotine version. Based on the different discrete methods, three versions, IPN (normal patterns), IPR (raster points), and IPM (MIM patterns), are implemented. To test the model's capabilities, we do not set a time limit while solving the model. Since CPLEX ran out of memory while initialing the model for all the instances in the third set except two, we only report the results of the instances in the first and second sets. There are two papers addressing 2D_U_SLOPP_D in the literature by Gonçalves and Wäscher (2020) and Luo et al. (2022); therefore, we compare our approaches with their three methods.

- **2DMIP_24h**: a mixed integer model by Gonçalves and Wäscher (2020).
- **BRKGA**: a biased random-key genetic algorithms by Gonçalves and Wäscher (2020).
- **BGA_VNS**: a biased genetic algorithm hybridized with VNS by Luo et al. (2022).

The 2DMIP_24h was solved using GUROBI OPTIMIZER version 6.5.2, and BRKGA was coded using the C ++ programming language. The BGA_VNS approach was coded using C++ and conducted on a computer with Intel(R) Xeon (R) CPU E5 2.4 GHz and 4 GB RAM running the Windows operating system. The experiments were conducted on a computer with an Intel E5-2630 @2.30 GHz CPU and 16 GB of physical memory, running the Linux operating system with Fedora release 24. The time limit of 2DMIP_24h was set to 24 h. 2DMIP_24h was tested on all instances of the first set and some instances of the second set. BRKGA and BGA_VNS were tested on all instances of the first and second sets. However, there are currently no results in the literature for the third set, as it is a constructed instance based on data from the 2018 ROADEF/EURO challenge (Libralesso and Fontan, 2021). To show the effect of dropping the guillotine cut, we also list again the results of DPR for comparison in the following.

The result on the first set of instances is shown in Table 4, where the column *OFV* means the objective function value and t (s) is the time in seconds per instance. The column $imp(\%)$ is the relative improvement of the results of the non-guillotine version over to the guillotine cut version, i.e., $100 * (OFV_{non} - OFV_g)/OFV_g$, where OFV_{non} and OFV_g are the objective function values of non-guillotine and guillotine version, respectively. We can see that our approaches IPN, IPR, and IPM find the optimal solution for each instance in generally less than 10 s, with one exception (#12) of 25 s. About 2DMIP_24h, it failed on all instances to find the optimal solutions within 24 h. BRKGA finds the optimal solution for 12 out of 14 instances (except #7 and #11), and BGA_VNS finds the optimal solution for all instances. However, the average computation time of BGA_VNS is 22.84 s, more than five times that of IPN. Therefore, our model is more effective than all of the three literature algorithms. Besides, the time used by IPR and IPM is about one-fourth with respect to IPN. Compared to the guillotine cut version, the result is improved for six instances, and the average improved ratio is 0.59%.

Table 3
Number of y-axis discrete points for the instances in the second set.

Size	n	Pure			$m=1$			$m=2$			$m=3$			$m=4$		
		$\lvert Y_0^n\rvert$	$\lvert Y_0^r\rvert$	$\lvert Y_0^m\rvert$	$\lvert Y_0^n\rvert$	$\lvert Y_0^r\rvert$	$\lvert Y_0^m\rvert$	$\lvert Y_0^n\rvert$	$\lvert Y_0^r\rvert$	$\lvert Y_0^m\rvert$	$\lvert Y_0^n\rvert$	$\lvert Y_0^r\rvert$	$\lvert Y_0^m\rvert$	$\lvert Y_0^n\rvert$	$\lvert Y_0^r\rvert$	$\lvert Y_0^m\rvert$
75*75	5	11.4	6.4	7.8	12.4	7.4	8.2	14.2	8.5	9.0	15.5	9.3	9.6	16.8	10.3	10.5
	10	24.4	14.3	15.4	25.1	15.6	16.1	26.3	16.4	16.8	28.0	17.4	17.7	29.2	18.4	18.5
	15	32.7	20.6	21.7	33.6	21.8	22.3	34.4	22.8	23.0	35.2	23.7	23.6	36.2	24.8	24.5
	20	39.0	28.2	28.9	39.4	29.2	29.2	40.1	30.0	29.9	40.9	31.0	30.8	41.6	31.9	31.6
	25	41.8	32.3	32.8	42.2	33.3	33.2	42.6	34.1	33.8	43.1	34.8	34.4	43.5	35.2	34.6
	avg.	29.9	20.4	21.3	30.5	21.5	21.8	31.5	22.4	22.5	32.5	23.2	23.2	33.5	24.1	24.0
112*50	5	9.4	6.1	6.6	10.2	6.7	7.1	11.4	7.3	7.6	12.4	8.1	8.2	13.5	9.0	8.9
	10	14.8	9.2	10.1	15.6	10.1	10.6	16.8	11.1	11.3	17.8	12.0	12.0	18.4	12.6	12.5
	15	21.8	15.0	15.8	22.7	16.0	16.3	23.2	16.8	16.6	23.8	17.6	17.1	24.4	18.4	17.7
	20	25.4	18.8	19.5	25.9	19.9	19.9	26.3	20.6	20.3	26.8	21.2	20.8	27.2	21.7	21.2
	25	27.4	21.6	21.9	27.6	22.2	22.1	28.1	22.7	22.6	28.5	23.2	23.0	28.8	23.7	23.3
	avg.	19.8	14.1	14.8	20.4	15.0	15.2	21.2	15.7	15.7	21.9	16.4	16.2	22.5	17.1	16.7
150*150	5	11.4	6.3	7.4	12.4	6.8	7.6	14.3	7.5	8.2	15.4	8.4	8.9	17.3	9.2	9.5
	10	28.6	13.6	15.2	31.0	14.7	16.2	32.8	15.7	16.9	34.6	16.8	17.8	36.8	17.9	18.8
	15	47.2	25.2	26.6	48.7	26.2	27.4	50.2	27.5	28.4	51.9	28.8	29.4	53.6	30.0	30.5
	20	58.3	30.4	32.8	59.9	31.8	33.4	61.1	33.2	34.8	62.5	34.6	36.0	64.0	36.0	37.2
	25	70.8	43.6	44.7	71.7	44.8	45.5	72.4	45.8	46.2	73.6	47.4	47.4	74.6	48.4	48.5
	avg.	43.3	23.8	25.3	44.7	24.9	26.0	46.2	25.9	26.9	47.6	27.2	27.9	49.3	28.3	28.9
225*100	5	7.8	4.7	5.3	9.2	5.4	5.8	10.2	6.0	6.3	11.1	6.6	6.9	12.3	7.3	7.5
	10	20.7	10.8	12.4	23.1	11.8	13.0	24.3	12.7	13.6	25.5	13.4	14.2	26.8	14.3	14.9
	15	29.2	15.2	16.6	30.9	16.2	17.3	32.1	17.3	18.2	33.0	18.1	18.8	34.2	19.2	19.7
	20	38.9	23.6	25.0	39.7	24.7	25.5	40.5	25.6	26.3	41.7	26.9	27.5	43.0	28.1	28.4
	25	46.2	29.2	30.3	46.9	30.1	30.8	47.5	30.8	31.2	48.3	32.2	32.4	49.3	33.4	33.2
	avg.	28.6	16.7	17.9	30.0	17.6	18.5	30.9	18.5	19.2	31.9	19.4	19.9	33.1	20.4	20.8
300*300	5	10.6	6.0	7.0	12.0	6.8	7.4	14.0	7.2	7.9	15.3	7.7	8.6	17.2	8.5	9.4
	10	38.3	16.1	18.5	42.9	17.2	19.6	45.2	18.4	20.5	48.2	19.6	21.4	50.4	20.8	22.4
	15	59.4	21.8	25.1	63.2	23.0	26.0	67.1	24.7	27.5	69.5	26.0	28.5	72.8	27.5	30.2
	20	90.8	38.1	42.6	92.9	39.4	43.3	96.4	41.2	44.9	99.4	43.0	46.3	102.4	45.9	48.5
	25	109.9	49.2	52.6	112.4	50.7	53.8	114.9	53.0	55.6	117.0	54.8	57.2	118.9	56.8	59.1
	avg.	61.8	26.3	29.2	64.7	27.4	30.0	67.5	28.9	31.3	69.9	30.2	32.4	72.3	31.9	33.9
450*200	5	8.2	5.2	5.8	9.4	5.7	6.3	10.9	6.4	6.7	12.3	7.2	7.4	14.3	7.8	8.1
	10	23.7	10.3	12.2	25.6	11.3	12.8	28.8	12.6	13.8	30.5	13.6	14.8	31.8	14.6	15.4
	15	47.2	20.0	22.3	49.6	21.1	23.1	51.6	22.4	24.0	54.0	24.0	25.3	56.4	25.5	26.6
	20	65.0	30.7	32.9	67.0	32.2	33.8	68.3	33.4	34.8	71.1	35.0	36.4	73.3	37.2	38.2
	25	73.2	34.6	37.5	74.4	36.4	38.6	76.8	38.1	39.8	79.2	39.8	41.2	80.6	41.4	42.8
	avg.	43.4	20.2	22.2	45.2	21.3	22.9	47.3	22.6	23.8	49.4	23.9	25.0	51.3	25.3	26.2
avg.		37.8	20.2	21.8	39.2	21.3	22.4	40.8	22.3	23.2	42.2	23.4	24.1	43.7	24.5	25.1

Table 4
Computational results for the first set instances (non-guillotine version).

#Inst.	2DMIP_24 h	BRKGA		BGA_VNS		IPN		IPR		IPM		DPR(guillotine)		
	OFV	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	$imp(\%)$
1	162	166	15.35	166	25.01	166	1.64	166	0.42	166	0.42	166	0.28	0.00
2	163	166	15.00	166	22.02	166	2.39	166	0.69	166	0.48	166	0.08	0.00
3	162	166	15.65	166	21.01	166	1.19	166	0.27	166	0.30	166	0.10	0.00
4	160	164	15.32	164	24.50	164	1.77	164	0.89	164	0.53	164	0.11	0.00
5	160	164	15.76	164	21.62	164	1.39	164	0.52	164	0.66	164	0.10	0.00
6	162	164	15.08	164	22.25	164	1.19	164	0.20	164	0.22	164	0.05	0.00
7	156	160	15.09	161	23.63	161	0.86	161	0.09	161	0.11	158	0.17	1.90
8	152	156	14.98	156	22.55	156	0.75	156	0.09	156	0.11	154	0.14	1.30
9	152	162	16.77	162	21.12	162	9.28	162	5.34	162	5.41	160	0.08	1.25
10	148	158	17.59	158	22.51	158	3.58	158	1.03	158	1.28	158	0.36	0.00
11	146	151	17.65	153	29.23	153	1.66	153	0.56	153	0.70	151	0.37	1.32
12	153	157	16.21	157	21.11	157	24.63	157	2.91	157	3.25	156	0.51	0.64
13	148	150	16.28	150	22.21	150	5.63	150	1.63	150	2.63	150	0.20	0.00
14	160	163	16.07	163	21.08	163	3.39	163	0.63	163	0.70	160	0.15	1.88
avg.	156.00	160.50	15.91	160.71	22.84	160.71	4.24	160.71	1.09	160.71	1.20	159.79	0.19	0.59

For the instances in the second set, since IPN ran out of memory for several instances, we only list the results of IPR and IPM in the following. Besides, since 2DMIP_24h failed to find an optimal solution within the time limit for most of the instances in the second set, and the total running time required to test all the instances in the second set is too long (Gonçalves and Wäscher, 2020), only a reduced set of instances from the second set is tested by Gonçalves and Wäscher (2020). Therefore, the results of 2DMIP_24h are also excluded. The results on the instances in the second set for categories 1, 2, and 3 are shown in Tables 5, 6, and 7, respectively. IPR and IPM found the optimal solution for all instances except five largest instances (in the class with $size = 450 * 200, n = 25, p^s = 10$) out of 5400 instances. CPLEX ran out of memory while initializing the model for these instances. The number of optimal solutions achieved by BRKGA and BGA_VNS in the second set is unreported since they are both heuristic algorithms, and the original papers do not detail the number of optimal solutions obtained. The results show that for the instances with small size ($n \leq 10$) both IPR and IPM found the optimal solutions in

Table 5
Computational results for the second set category 1 instances (non-guillotine version).

Class	Size	n	p^s	BRKGA		BGA_VNS		IPR		IPM		DPR(guillotine)		
				OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	imp(\%)
1	75*75	5	6	3697.87	6.04	3697.87	17.72	3697.87	0.01	3697.87	0.00	3694.32	0.02	0.09
2		10	6	4277.03	10.58	4282.95	20.54	4282.95	0.20	4282.95	0.03	4256.02	0.03	0.62
3		15	6	4576.88	14.56	4578.73	35.25	4578.73	0.37	4578.73	0.07	4566.10	0.08	0.28
4		20	6	4634.48	18.78	4635.85	24.23	4636.75	5.26	4636.75	2.81	4615.43	0.15	0.47
5		25	6	4726.58	21.81	4734.18	24.63	4736.07	3.25	4736.07	1.39	4694.02	0.22	0.89
6		5	8	3701.70	6.88	3701.70	23.45	3701.70	0.01	3701.70	0.01	3683.23	0.02	0.43
7		10	8	4411.05	12.66	4411.70	26.76	4412.02	0.64	4412.02	0.11	4386.17	0.09	0.57
8		15	8	4641.70	16.72	4647.50	37.54	4648.80	1.61	4648.80	0.59	4613.23	0.19	0.76
9		20	8	4925.37	21.82	4930.37	35.05	4934.35	7.61	4934.35	3.76	4883.78	0.42	1.05
10		25	8	4862.13	24.54	4865.97	32.40	4868.40	13.27	4868.40	8.51	4835.60	0.68	0.71
11		5	10	4097.03	9.19	4099.33	37.94	4099.33	0.03	4099.33	0.01	4083.57	0.04	0.37
12		10	10	4722.27	16.45	4726.15	41.15	4732.27	4.21	4732.27	2.02	4710.92	0.32	0.44
13		15	10	4867.80	21.60	4873.77	53.52	4876.08	5.10	4876.08	1.76	4845.17	0.41	0.63
14		20	10	4946.43	25.01	4949.93	44.13	4963.08	77.11	4963.08	64.06	4928.65	1.35	0.72
15		25	10	4987.70	28.97	4995.83	43.87	5006.28	51.96	5006.28	43.29	4968.50	1.41	0.77
avg.				4538.40	17.04	4542.12	33.21	4544.98	11.38	4544.98	8.56	4517.65	0.00	0.59
16	112*50	5	6	3548.87	5.81	3548.87	18.47	3550.12	0.01	3550.12	0.01	3530.42	9.93	0.54
17		10	6	4348.62	10.76	4348.62	19.55	4349.67	0.30	4349.67	0.09	4339.18	0.05	0.22
18		15	6	4512.40	14.61	4513.37	30.16	4513.37	0.55	4513.37	0.15	4495.20	0.07	0.39
19		20	6	4642.50	18.99	4651.35	23.46	4652.92	2.11	4652.92	0.73	4618.17	0.17	0.76
20		25	6	4715.23	22.43	4716.42	23.77	4718.15	6.58	4718.15	4.34	4686.22	0.23	0.68
21		5	8	3859.63	7.71	3860.48	28.01	3860.48	0.01	3860.48	0.00	3829.67	0.02	0.82
22		10	8	4562.03	13.06	4573.20	30.48	4573.33	0.36	4573.33	0.14	4536.90	0.11	0.82
23		15	8	4829.83	19.43	4832.43	36.08	4834.53	3.08	4834.53	1.01	4792.70	0.32	0.88
24		20	8	4722.65	21.41	4724.28	28.24	4728.03	6.61	4728.03	2.85	4706.17	0.33	0.46
25		25	8	4811.43	24.58	4814.52	31.06	4820.28	10.87	4820.28	5.59	4791.00	0.54	0.60
26		5	10	4151.63	8.80	4153.23	29.51	4153.23	0.12	4153.23	0.02	4138.25	0.06	0.32
27		10	10	4411.72	13.13	4418.35	39.14	4420.85	0.45	4420.85	0.09	4396.10	0.12	0.55
28		15	10	4730.77	19.20	4737.73	40.12	4739.50	2.69	4739.50	1.01	4688.12	0.38	1.10
29		20	10	4925.95	23.92	4929.42	37.03	4936.90	16.72	4936.90	13.56	4895.85	0.69	0.85
30		25	10	5005.03	28.40	5007.33	41.74	5019.82	39.40	5019.82	27.47	4988.35	0.95	0.65
avg.				4518.55	16.82	4521.97	35.45	4524.75	5.99	4524.75	3.80	4495.49	0.93	0.64

Table 6
Computational results for the second set category 2 instances (non-guillotine version).

Class	Size	n	p^s	BRKGA		BGA_VNS		IPR		IPM		DPR(guillotine)		
				OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	imp(\%)
1	150*150	5	6	14910.83	6.73	14910.82	26.24	14910.82	0.00	14910.82	0.01	14794.82	0.05	0.77
2		10	6	15832.06	9.78	15844.15	18.98	15844.15	0.03	15844.15	0.03	15772.18	0.10	0.44
3		15	6	17386.47	13.17	17400.52	20.16	17400.52	0.05	17400.52	0.07	17356.57	0.14	0.26
4		20	6	18383.32	17.82	18390.23	22.96	18390.23	0.88	18390.23	1.13	18256.58	0.39	0.73
5		25	6	19000.23	22.51	19010.83	25.30	19016.53	4.31	19016.53	5.72	18824.05	0.98	1.06
6		5	8	14429.35	6.63	14429.33	26.84	14429.33	0.00	14429.33	0.01	14384.88	0.05	0.32
7		10	8	17855.42	13.92	17889.88	32.77	17895.67	3.70	17895.67	3.64	17780.28	0.52	0.67
8		15	8	18851.20	17.97	18878.25	30.30	18878.72	0.98	18878.72	1.33	18747.35	0.86	0.69
9		20	8	19425.53	22.76	19446.45	35.76	19464.73	10.47	19464.73	11.75	19315.80	2.04	0.79
10		25	8	19619.03	26.51	19637.15	35.94	19667.50	517.65	19667.50	372.64	19530.53	5.37	0.72
11		5	10	15244.85	7.62	15252.87	29.32	15252.87	0.01	15252.87	0.01	15174.78	0.06	0.51
12		10	10	18939.72	16.30	18953.63	41.69	18960.83	0.80	18960.83	0.91	18839.87	1.04	0.66
13		15	10	18911.88	19.68	18937.30	39.47	18954.85	9.21	18954.85	10.63	18747.58	1.82	1.11
14		20	10	19575.43	22.48	19594.07	36.11	19614.82	59.03	19614.82	48.54	19438.30	3.53	0.91
15		25	10	19900.18	30.80	19920.70	48.98	19987.50	423.70	19987.50	324.17	19822.68	10.18	0.85
avg.				17884.37	16.98	17899.75	31.39	17911.27	68.72	17911.27	52.04	17785.75	1.81	0.70
16	225*100	5	6	13636.08	5.97	13655.67	23.68	13655.67	0.00	13655.67	0.01	13611.92	0.04	0.29
17		10	6	17022.84	10.49	17029.72	23.15	17029.72	0.03	17029.72	0.05	16938.10	0.09	0.53
18		15	6	17290.72	13.03	17291.08	20.41	17291.08	0.14	17291.08	0.19	17203.60	0.16	0.49
19		20	6	18740.97	17.38	18775.42	25.58	18775.42	1.81	18775.42	2.41	18606.03	0.65	0.90
20		25	6	18815.77	22.04	18846.90	26.78	18850.28	17.48	18850.28	16.70	18708.25	1.29	0.77
21		5	8	15222.01	6.75	15245.35	28.53	15245.35	0.01	15245.35	0.01	15217.80	0.05	0.18
22		10	8	17845.32	13.01	17859.05	32.67	17860.95	0.70	17860.95	0.88	17720.82	0.33	0.77
23		15	8	18735.07	16.32	18747.93	32.11	18751.03	1.03	18751.03	1.15	18620.30	0.74	0.72
24		20	8	19104.65	22.00	19121.05	34.70	19126.82	3.78	19126.82	4.79	18999.55	1.74	0.68
25		25	8	19500.98	25.90	19525.05	41.71	19549.60	73.86	19549.60	69.26	19382.40	4.29	0.88
26		5	10	15634.70	7.76	15655.92	29.56	15655.92	0.01	15655.92	0.01	15542.52	0.08	0.67
27		10	10	17503.34	13.39	17515.47	37.86	17515.47	0.28	17515.47	0.34	17366.72	0.52	0.89
28		15	10	19078.07	18.57	19079.80	39.17	19112.78	12.22	19112.78	11.85	18913.72	2.45	1.08
29		20	10	19688.90	25.24	19738.47	40.27	19779.23	135.30	19779.23	118.60	19632.15	6.66	0.76
30		25	10	19810.03	27.57	19835.90	43.91	19871.62	46.94	19871.62	50.40	19707.63	4.99	0.84
avg.				17841.96	16.36	17861.52	32.01	17871.40	19.57	17871.40	18.44	17744.77	1.61	0.70

Table 7
Computational results for the second set category 3 instances (non-guillotine version).

Class	Size	n	p^s	BRKGA		BGA_VNS		IPR		IPM		DPR(guillotine)		
				OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	imp(\%)
1		5	6	60110.69	6.48	60151.75	20.52	60151.75	0.01	60151.75	0.01	59589.37	1.57	0.82
2		10	6	65872.74	9.68	65900.17	16.75	65900.17	0.06	65900.17	0.08	65763.73	1.55	0.21
3		15	6	69915.07	13.32	70004.52	20.84	70004.52	0.16	70004.52	0.23	69424.47	2.10	0.85
4		20	6	72935.98	17.90	72990.33	24.39	72990.33	1.38	72990.33	2.86	72494.38	3.53	0.67
5		25	6	75630.38	22.09	75657.70	24.53	75676.20	15.02	75676.20	19.23	75017.87	4.03	0.90
6		5	8	57690.52	6.11	57717.85	18.27	57717.85	0.00	57717.85	0.01	57585.43	1.42	0.25
7		10	8	69352.93	12.43	69382.80	27.78	69418.83	6.72	69418.83	7.39	69057.08	3.84	0.51
8	300*300	15	8	74210.50	17.27	74347.35	31.73	74347.35	1.96	74347.35	3.01	73679.27	4.66	0.91
9		20	8	76016.22	21.72	76128.12	32.93	76195.95	20.65	76195.95	30.02	75528.63	9.70	0.90
10		25	8	77620.77	26.00	77690.70	38.34	77798.12	185.01	77798.12	194.08	77178.77	19.31	0.82
11		5	10	60054.53	7.25	60119.03	24.81	60119.03	0.01	60119.03	0.02	59970.58	1.66	0.20
12		10	10	73493.90	14.32	73654.18	30.14	73663.80	0.68	73663.80	1.01	73352.73	4.09	0.44
13		15	10	74343.53	18.71	74495.17	37.91	74510.63	7.66	74510.63	7.88	74058.07	6.32	0.59
14		20	10	79091.05	26.72	79193.23	50.79	79401.48	1853.27	79401.48	1830.90	78722.25	33.85	0.88
15		25	10	79331.70	30.33	79418.42	54.19	79675.70	1485.83	79675.70	1543.93	78921.28	42.22	0.97
avg.				71044.70	16.69	71123.42	30.26	71171.45	238.56	71171.45	242.71	70689.59	9.32	0.66
16		5	6	55585.71	5.81	55647.45	15.12	55647.45	0.01	55647.45	0.01	55345.02	1.11	0.53
17		10	6	68363.29	10.56	68428.60	22.49	68428.60	0.05	68428.60	0.08	68157.33	1.93	0.39
18		15	6	72802.43	15.14	72846.10	26.51	72846.10	0.30	72846.10	0.47	72302.37	2.51	0.73
19		20	6	72508.94	17.60	72505.50	24.18	72528.08	0.44	72528.08	0.67	71964.07	2.75	0.77
20		25	6	76146.53	23.10	76254.90	27.50	76259.48	6.12	76259.48	9.76	75596.92	5.60	0.87
21		5	8	58947.60	6.17	58967.98	18.08	58967.98	0.01	58967.98	0.01	58758.78	1.10	0.34
22		10	8	68540.91	12.05	68575.20	24.81	68628.90	2.08	68628.90	2.69	68186.45	2.73	0.60
23	450*200	15	8	73547.05	17.23	73636.38	33.90	73649.48	11.07	73649.48	11.02	73113.17	4.90	0.76
24		20	8	77835.85	22.73	78028.68	37.82	78072.57	43.68	78072.57	53.42	77282.15	9.46	1.04
25		25	8	78806.68	26.76	78903.28	41.84	78986.98	852.81	78986.98	381.98	78269.40	21.34	0.93
26		5	10	65034.44	8.71	65194.35	29.55	65194.35	0.01	65194.35	0.02	64746.97	1.90	0.62
27		10	10	68596.42	13.18	68697.13	31.94	68698.37	0.24	68698.37	0.40	67936.45	3.09	1.14
28		15	10	76776.17	19.32	76850.12	45.31	76907.53	22.72	76907.53	20.61	76180.58	9.69	0.99
29		20	10	78587.83	25.98	78755.95	51.13	78960.42	543.16	78960.42	606.23	78203.08	29.09	0.99
30		25	10	78554.79	29.16	78655.82	54.25	**78240.93**	**2117.38**	**78222.17**	**1887.15**	78254.72	52.67	–
avg.				71375.64	16.90	71463.16	32.29	71467.82	240.01	71466.56	198.30	70953.16	9.99	0.76

Table 8
Computational results for the first set instances (guillotine version).

#Inst.	DP-B&B	B&B-ANDOR		DPD		IHDP		DPC		DPN		DPR		DPM	
	OFV	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)
1	166	166	4.6	166	18.9	166	0.9	166	316	166	1.21	166	0.28	166	0.15
2	160	160	3.6	166	16.4	166	0.7	166	272	166	0.49	166	0.08	166	0.06
3	162	162	4.4	166	16.5	166	0.7	166	267	166	0.86	166	0.10	166	0.08
4	158	160	6.2	164	18.3	164	0.3	164	218	164	0.52	164	0.11	164	0.07
5	164	164	13.5	164	76.9	164	0.3	164	215	164	1.25	164	0.10	164	0.09
6	164	164	1.3	164	0.9	164	0.5	164	172	164	0.16	164	0.05	164	0.04
7	157	158	12.5	158	0.8	158	0.2	158	389	158	0.87	158	0.17	158	0.06
8	154	154	8.1	154	1.2	154	0.1	154	387	154	0.98	154	0.14	154	0.05
9		153		160	14.3	160	0.9	160	30	160	0.54	160	0.08	160	0.06
10		148		158	2.2	158	0.5	158	277	158	1.91	158	0.36	158	0.20
11		143		151	26.8	151	6.3	151	417	151	3.33	151	0.37	151	0.29
12		150		156	1126	156	119	156	293	156	4.32	156	0.51	156	0.33
13		142		150	9.1	150	2.2	150	143	150	1.44	150	0.20	150	0.06
14		160		160	1.0	160	0.1	160	107	160	0.55	160	0.15	160	0.06
avg(1-8)	160.63	161.00	6.8	162.75	18.7	162.75	0.5	162.75	280	162.75	0.79	162.75	0.13	162.75	0.07
avg(1-14)		156.33		159.79	95.0	159.79	9.5	159.79	250	159.79	1.32	159.79	0.19	159.79	0.11

a short time (less than 10 s). However, the time increases significantly as n increases. In addition, the time also increases with the size of the sheet and the p^s value. For the largest instances, the average time increases to 2117.38 s. The average time for BRKGA is 16.80 s, and for BGA_VNS, it is 32.44 s, roughly twice of BRKGA. However, the BGA_VNS obtains better average objective values than BRKGA. For the second set category 1 instances, the average time of IPR and IPM is less than that of BRKGA and BGA_VNS. For the instances with size=225*100 in the second set category 2, the average time of IPR and IPM is slightly larger than BRKGA, which is about 50% of the BGA_VNS. For other instances, the average time of IPR and IPM is greater than that of BRKGA and BGA_VNS.

6.4. Results for the 2D_UG_SLOPP_D

To analyze the performance of our approach, we implemented four versions of the DP approach. The first version, DPC, considers all integer positions for vertical and horizontal guillotine cuts, while the second version, DPN, considers the normal patterns. The third version,

Table 9
Computational results for the second set category 1 instances (guillotine version).

Class	size	n	p^s	DPD		IHDP		B&BC		DPC		DPN		DPR		DPM	
				OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV		OFV	t (s)	OFV	t (s)	OFV	t (s)
1		5	6	3694.3	3.4	3694.3	0.4	3694.3	0.1	3694.3	29.2	3694.3	0.08	3694.3	0.02	3694.3	0.01
2		10	6	4229.9	4.5	4256.0	2.5	4256.0	15.2	4256.0	32.5	4256.0	0.35	4256.0	0.03	4256.0	0.02
3		15	6	4540.4	5.4	4566.1	3.9	4566.1	1.6	4566.1	42.8	4566.1	0.72	4566.1	0.08	4566.1	0.05
4		20	6	4593.4	5.8	4615.4	5.3	4605.7	42.8	4615.4	41.9	4615.4	1.25	4615.4	0.15	4615.4	0.15
5		25	6	4674.7	6.2	4694.0	6.3	4691.4	56.7	4694.0	55.8	4694.0	1.64	4694.0	0.22	4694.0	0.30
6		5	8	3638.0	2.9	3683.2	0.6	3683.2	0.9	3683.2	29.6	3683.2	0.16	3683.2	0.02	3683.2	0.01
7		10	8	4375.5	4.7	4386.2	3.2	4372.0	62.0	4386.2	40.8	4386.2	0.74	4386.2	0.09	4386.2	0.06
8	75*75	15	8	4585.3	6.1	4613.2	5.3	4604.2	78.0	4613.2	45.8	4613.2	1.20	4613.2	0.19	4613.2	0.16
9		20	8	4876.4	6.3	4883.8	6.8	4850.7	150.2	4883.8	71.1	4883.8	2.30	4883.8	0.42	4883.8	0.49
10		25	8	4826.9	7.8	4835.6	7.5	4792.7	91.9	4835.6	51.0	4835.6	2.86	4835.6	0.68	4835.6	1.16
11		5	10	4055.1	3.2	4083.6	1.1	4001.6	16.6	4083.6	35.7	4083.6	0.35	4083.6	0.04	4083.6	0.02
12		10	10	4686.6	5.7	4710.9	5.4	4637.4	107.7	4710.9	50.2	4710.9	1.82	4710.9	0.32	4710.9	0.20
13		15	10	4826.7	6.7	4845.2	6.3	4678.9	122.6	4845.2	54.7	4845.2	2.01	4845.2	0.41	4845.2	0.38
14		20	10	4921.0	9.4	4928.7	8.2	4629.1	155.8	4928.7	75.7	4928.7	4.00	4928.7	1.35	4928.7	1.75
15		25	10	4960.2	10.2	4968.5	9.4	4783.5	217.9	4968.5	61.2	4968.5	4.66	4968.5	1.41	4968.5	2.67
16		5	6	3480.6	2.6	3530.4	0.3	3530.4	0.0	3530.4	20.5	3530.4	0.05	3530.4	0.01	3530.4	0.00
17		10	6	4305.4	4.3	4339.2	2.5	4339.2	20.4	3829.7	29.6	3829.7	0.42	3829.7	0.05	4339.2	0.02
18		15	6	4470.8	4.3	4495.2	4.1	4495.2	13.4	4138.3	31.6	4138.3	0.62	4138.3	0.07	4495.2	0.05
19		20	6	4607.8	5.8	4618.2	5.8	4611.7	59.5	4339.2	28.0	4339.2	1.33	4339.2	0.17	4618.2	0.16
20		25	6	4679.7	6.8	4686.2	6.8	4671.7	82.3	4536.9	41.5	4536.9	1.68	4536.9	0.23	4686.2	0.35
21		5	8	3752.3	2.0	3829.7	0.7	3829.7	7.8	4396.1	39.7	4396.1	0.11	4396.1	0.02	3829.7	0.01
22		10	8	4523.0	4.6	4536.9	4.0	4536.9	26.3	4495.2	40.1	4495.2	0.83	4495.2	0.11	4536.9	0.06
23	112*50	15	8	4766.9	5.6	4792.7	6.1	4770.5	130.3	4792.7	61.3	4792.7	1.89	4792.7	0.32	4792.7	0.25
24		20	8	4685.9	5.9	4706.2	6.8	4681.5	74.4	4688.1	56.3	4688.1	1.99	4688.1	0.33	4706.2	0.36
25		25	8	4772.0	8.1	4791.0	7.9	4715.3	122.0	4618.2	47.3	4618.2	2.75	4618.2	0.54	4791.0	0.81
26		5	10	4076.5	3.6	4138.3	1.0	4138.3	11.5	4706.2	50.6	4706.2	0.28	4706.2	0.06	4138.3	0.02
27		10	10	4357.6	4.5	4396.1	3.9	4374.3	70.6	4895.9	56.6	4895.9	0.79	4895.9	0.12	4396.1	0.08
28		15	10	4661.9	5.7	4688.1	6.3	4590.7	150.0	4686.2	47.4	4686.2	1.88	4686.2	0.38	4688.1	0.31
29		20	10	4891.4	8.7	4895.9	8.4	4705.4	154.6	4791.0	64.9	4791.0	2.97	4791.0	0.69	4895.9	0.93
30		25	10	4975.2	9.3	4988.4	8.9	4856.0	206.3	4988.4	83.0	4988.4	4.08	4988.4	0.95	4988.4	1.58
avg.				4483.0	5.7	4506.6	4.9	4456.4	75.0	4506.6	47.2	4506.6	1.53	4506.6	0.32	4506.6	0.41

Table 10
Computational results for the second set category 2 instances (guillotine version).

Class	size	n	p^s	DPD		IHDP		DPC		DPN		DPR		DPM	
				OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)	OFV	t (s)
1		5	6	14546.7	24.7	14794.8	7.9	14794.8	309.5	14794.8	0.3	14794.8	0.1	14794.8	0.1
2		10	6	15544.8	35.7	15772.2	42.1	15772.2	751.1	15772.2	1.6	15772.2	0.1	15772.2	0.1
3		15	6	17263.9	35.8	17356.6	77.2	17356.6	801.6	17356.6	2.7	17356.6	0.1	17356.6	0.1
4		20	6	18128.6	58.6	18256.6	138.5	18256.6	1174.2	18256.6	7.8	18256.6	0.4	18256.6	0.3
5		25	6	18716.7	69.2	18701.4	163.1	18824.1	1400.6	18824.1	16.3	18824.1	1.0	18824.1	1.1
6		5	8	14276.1	27.3	14384.9	4.9	14384.9	277.1	14384.9	0.2	14384.9	0.0	14384.9	0.0
7		10	8	17632.8	40.9	17780.3	74.94	17780.3	1142.4	17780.3	6.1	17780.3	0.5	17780.3	0.3
8	150*150	15	8	18615.3	72.1	18747.4	136.4	18747.4	1210.0	18747.4	12.4	18747.4	0.9	18747.4	0.6
9		20	8	19193.3	73.6	19315.8	185.8	19315.8	1596.7	19315.8	22.8	19315.8	2.0	19315.8	2.2
10		25	8	19435.9	131.2	19530.5	184.8	19530.5	1671.0	19530.5	47.2	19530.5	5.4	19530.5	6.3
11		5	10	15067.9	24.0	15174.8	9.2	15174.8	756.1	15174.8	0.4	15174.8	0.1	15174.8	0.0
12		10	10	18776.0	63.5	18839.9	117.0	18839.9	1266.1	18839.9	10.3	18839.9	1.0	18839.9	0.6
13		15	10	18685.0	71.9	18611.3	153.1	18747.6	1331.3	18747.6	18.5	18747.6	1.8	18747.6	1.3
14		20	10	19372.4	99.7	19438.3	187.3	19438.3	1436.1	19438.3	32.9	19438.3	3.5	19438.3	3.1
15		25	10	19724.6	137.1	19822.7	217.0	19822.7	1807.2	19822.7	62.3	19822.7	10.2	19822.7	14.6
16		5	6	13420.9	21.3	13611.9	6.1	13611.9	605.2	13611.9	0.2	13611.9	0.0	13611.9	0.0
17		10	6	16709.1	43.0	16938.1	84.6	16938.1	1086.7	16938.1	1.9	16938.1	0.1	16938.1	0.1
18		15	6	17058.2	45.4	17203.6	96.3	17203.6	423.1	17203.6	2.8	17203.6	0.2	17203.6	0.2
19		20	6	18475.8	63.0	18606.0	141.7	18606.0	554.2	18606.0	9.2	18606.0	0.6	18606.0	0.6
20		25	6	18591.3	74.7	18708.3	190.4	18708.3	1390.6	18708.3	19.8	18708.3	1.3	18708.3	1.6
21		5	8	15024.3	23.3	15217.8	15.6	15217.8	749.1	15217.8	0.3	15217.8	0.0	15217.8	0.0
22		10	8	17541.9	53.7	17720.8	163.5	17720.8	1061.2	17720.8	5.3	17720.8	0.3	17720.8	0.2
23	225*100	15	8	18491.1	63.0	18518.1	143.5	18620.3	582.1	18620.3	10.3	18620.3	0.7	18620.3	0.5
24		20	8	18814.5	84.3	18999.6	210.4	18999.6	645.0	18999.6	22.7	18999.6	1.7	18999.6	1.7
25		25	8	19215.2	109.9	19382.4	217.2	19382.4	1727.3	19382.4	38.4	19382.4	4.3	19382.4	6.5
26		5	10	15248.2	35.2	15542.5	31.8	15542.5	849.0	15542.5	0.6	15542.5	0.1	15542.5	0.1
27		10	10	17191.7	47.8	17366.7	106.5	17366.7	515.1	17366.7	6.8	17366.7	0.5	17366.7	0.4
28		15	10	18733.6	92.6	18913.7	192.1	18913.7	709.6	18913.7	20.2	18913.7	2.5	18913.7	1.6
29		20	10	19537.9	131.8	19513.5	220.0	19632.2	1502.5	19632.2	50.4	19632.2	6.7	19632.2	7.5
30		25	10	19561.2	122.9	19707.6	230.3	19707.6	1696.9	19707.6	46.1	19707.6	5.0	19707.6	7.5
avg.				17619.8	65.9	17749.3	125.0	17765.3	1034.3	17765.3	15.9	17765.3	1.7	17765.3	2.0

Table 11
Computational results for the second set category 3 instances (guillotine version).

Class	size	n	p^s	DPN		DPR		DPM	
				OFV	t (s)	OFV	t (s)	OFV	t (s)
1		5	6	59589.4	1.8	59589.4	1.6	59589.4	1.9
2		10	6	65763.7	7.2	65763.7	1.6	65763.7	2.3
3		15	6	69424.5	16.8	69424.5	2.1	69424.5	2.6
4		20	6	72494.4	70.7	72494.4	3.5	72494.4	4.4
5		25	6	75017.9	100.6	75017.9	4.0	75017.9	5.2
6		5	8	57585.4	1.8	57585.4	1.4	57585.4	2.4
7		10	8	69057.1	51.3	69057.1	3.8	69057.1	8.8
8	300*300	15	8	73679.3	83.4	73679.3	4.7	73679.3	5.4
9		20	8	75528.6	191.4	75528.6	9.7	75528.6	9.4
10		25	8	77178.8	395.7	77178.8	19.3	77178.8	25.2
11		5	10	59970.6	5.0	59970.6	1.7	59970.6	3.3
12		10	10	73352.7	40.3	73352.7	4.1	73352.7	4.6
13		15	10	74058.1	124.6	74058.1	6.3	74058.1	6.7
14		20	10	78722.3	490.7	78722.3	33.9	78722.3	50.9
15		25	10	78921.3	774.2	78921.3	42.2	78921.3	70.2
16		5	6	55345.0	1.5	55345.0	1.1	55345.0	1.4
17		10	6	68157.3	9.5	68157.3	1.9	68157.3	2.4
18		15	6	72302.4	32.7	72302.4	2.5	72302.4	3.0
19		20	6	71964.1	76.4	71964.1	2.8	71964.1	3.4
20		25	6	75596.9	378.8	75596.9	5.6	75596.9	7.1
21		5	8	58758.8	1.7	58758.8	1.1	58758.8	1.5
22		10	8	68186.5	24.6	68186.5	2.7	68186.5	3.2
23	450*200	15	8	73113.2	95.5	73113.2	4.9	73113.2	5.4
24		20	8	77282.2	371.1	77282.2	9.5	77282.2	10.7
25		25	8	78269.4	1376.8	78269.4	21.3	78269.4	30.3
26		5	10	64747.0	4.9	64747.0	1.9	64747.0	2.3
27		10	10	67936.5	34.3	67936.5	3.1	67936.5	3.3
28		15	10	76180.6	507.8	76180.6	9.7	76180.6	9.3
29		20	10	78203.1	715.6	78203.1	29.1	78203.1	37.7
30		25	10	78254.7	1080.1	78254.7	52.7	78254.7	66.5
avg.				70821.4	235.6	70821.4	9.7	70821.4	13.0

Table 12
Computational results for the third set of subset A instances (guillotine version).

Inst.	n	m	DPN		DPR		DPM	
			OFV	t (s)	OFV	t (s)	OFV	t (s)
A1	5	5	46267	195.15	46267	13.08	46267	11.69
A2	72	2	47988	2537.83	47988	1585.32	47988	1547.00
A3	68	0	48000	0.19	48000	2.12	48000	0.10
A4	68	4	47946	1391.97	47946	386.16	47946	266.08
A5	97	2	47976	250.98	47976	153.99	47976	120.35
A6	37	2	47880	188.11	47880	24.77	47880	19.12
A7	57	1	47944	2.65	47944	2.33	47944	0.83
A8	129	1	47987	480.64	47987	76.48	47987	54.87
A9	63	3	47963	1956.76	47963	697.53	47963	688.30
A10	086	0	48000	0.12	48000	1.35	48000	0.07
A11	86	7	47744	3388.00	47744	1817.75	47744	1653.91
A12	50	0	48000	0.09	48000	4.44	48000	0.05
A13	272	2	47872	173.54	47872	65.10	47872	30.62
A14	361	0	48000	0.18	48000	5.63	48000	0.26
A15	392	2	47976	2622.67	47976	1626.60	47976	1516.30
A16	38	1	47926	60.36	47926	25.99	47926	12.46
A17	21	3	47704	504.84	47704	102.80	47704	48.68
A18	73	3	47879	2359.76	47879	773.08	47879	734.84
A19	47	0	48000	0.06	48000	1.89	48000	0.02
A20	17	0	48000	0.02	48000	0.86	48000	0.01
avg.			47853	806	47853	368	47853	335

called DPR, considers the raster points. The last version, DPM, considers the MIM patterns. Note that Afsharian et al. (2014) also implemented the same DPC to calculate the optimal solutions for the test instance. However, the authors did not report the detailed results of each instance. To verify the correctness of our approaches and compare these approaches fairer, we implemented DPC and ran it on our machine for all used instances.

To the best of our knowledge, there is no other exact approach except DPC and B&BC (Martin et al., 2020) in the literature. Therefore, we compare our approach with the following approaches (all heuristic except B&BC, but this becomes heuristic in case of a time limit):

- **DP-B&B**: the DP-based heuristic including a branch-and-bound search proposed by Carnieri et al. (1993).
- **B&B-ANDOR**: the branch-and-bound algorithm based on an AND /OR-graph approach by Vianna and Arenales (2006). Here, we do not use the improved version of **B&B-ANDOR** by Neidlein et al. (2009) because Neidlein et al. (2009) only test their approach on different instances constructed randomly.
- **DPD**: the DP-based heuristic approach proposed by Afsharian et al. (2014).

Table 13
Computational results for the third set of subset B instances (guillotine version).

Inst.	n	m	DPN		DPR		DPM	
			OFV	t (s)	OFV	t (s)	OFV	t (s)
B1	68	1	47971	231.60	47971	63.04	47971	59.64
B2	383	2	47980	3019.24	47980	2027.06	47980	1986.55
B3	332	0	48000	0.15	48000	3.30	48000	0.27
B4	261	1	47988	90.26	47988	78.32	47988	39.22
B5	207	5	47622	2584.09	47622	538.27	47622	464.45
B6	204	0	48000	0.18	48000	3.64	48000	0.18
B7	241	4	47959	533.91	47959	224.64	47959	216.37
B8	334	0	48000	0.18	48000	2.50	48000	0.20
B9	247	1	47989	108.75	47989	69.75	47989	32.23
B10	214	2	47883	527.51	47883	103.37	47883	97.17
B11	274	8	47795	4340.18	47795	2306.20	47795	2177.39
B12	439	6	47955	5167.24	47955	3231.50	47955	3128.70
B13	656	5	47948	3664.57	47948	2064.65	47948	1895.63
B14	267	1	47992	48.58	47992	43.99	47992	38.83
B15	431	0	48000	0.24	48000	5.29	48000	0.38
avg.			47939	1354	47939	718	47939	676

Table 14
Computational results for the third set of subset X instances (guillotine version).

Inst.	n	m	DPN		DPR		DPM	
			OFV	t (s)	OFV	t (s)	OFV	t (s)
X1	300	2	47978	3146.99	47978	1600.13	47978	1572.96
X2	247	1	47989	243.96	47989	77.72	47989	36.54
X3	258	0	48000	0.12	48000	2.68	48000	0.16
X4	371	4	47956	2627.18	47956	1354.86	47956	1218.42
X5	124	5	47950	2055.08	47950	1206.56	47950	1126.74
X6	412	4	47959	3244.52	47959	2316.68	47959	2088.10
X7	215	1	47981	53.46	47981	34.94	47981	16.97
X8	173	4	47937	3192.35	47937	1707.11	47937	1622.45
X9	233	0	48000	0.14	48000	0.78	48000	0.14
X10	375	4	47967	2160.67	47967	981.28	47967	941.55
X11	362	8	47730	4024.45	47730	2253.67	47730	1880.84
X12	302	3	47967	1896.91	47967	1358.73	47967	1231.43
X13	344	3	47967	1269.65	47967	565.92	47967	483.08
X14	253	3	47959	2036.12	47959	964.27	47959	864.63
X15	296	1	47984	1415.07	47984	829.90	47984	802.54
avg.			47955	1824	47955	1017	47955	926

- **IHDP**: the improved heuristic-dynamic programming approach proposed by Yin et al. (2019).
- **B&BC**: the Benders decomposition proposed by Martin et al. (2020). Note that this approach is proposed for the constrained version (the available number of each item type is limited) of our problem, and the authors also tested their approach for the unconstrained case.

Since we cannot get the original papers of DP-B&B and B&B-ANDOR, we only list the computer environment for the recently proposed approaches DPD, IHDP and B&BC. DPD was encoded in C#, Version 4.0, and ran on a PC with an AMD E-350@ processor with 4 GB of RAM and a clock speed of 1.60 GHz under the Windows Operating System. IHDP was encoded in C/C++, and ran on a PC with an Intel Core i7cpu@360HZ and 8 GB of RAM. B&BC was coded in C++ and carried out on a PC with an Intel Core i7-3770 (3.40 GHz) processor and 16 GB of RAM under the Ubuntu 16.04 Operating System. The time limit for B&BC is set to 300 s in Martin et al. (2020), hence it did not find all optimum values. The experimental results for DP-B&B and B&B-ANDOR were obtained from the paper by Afsharian et al. (2014), where only the results of instances with a single defect in the first set is given for DP-B&B and the results for all instances in the first set are given for B&B-ANDOR. BBC was tested in the second set category 1 instances. DPD and IHDP were tested on all instances in the first set and the first two categories of instances in the second set. The layouts obtained by our approach for some selected instances are given in Appendix.

6.4.1. Results for the first set

The comparative results between DP-B&B, B&B-ANDOR, DPD, DPC, DPN, DPR, and DPM for the first set of instances are shown in Table 8. In the paper by Afsharian et al. (2014), the computation time is not reported for DP-B&B, and it is only provided for the first 8 instances with a single defect for B&B-ANDOR. Therefore, only the OFV value for the first 8 instances is reported for DP-B&B. Among all the heuristic approaches DP-B&B, B&B-ANDOR, DPD and IHDP, the two ones finding the optimal solution for all instances are DPD and IHDP. The average time of IHDP is 9.5 s which is a tenth of DPD. Our proposed approaches, DPN, DPR, and DPM, found the optimal solution in a short time. The average time spent by DPC is about 250 s, and the maximum time cost is 417 s. The average time DPN uses is 1.32 s, which is about 0.52% with respect to DPC. Moreover, DPR further reduces the average time to 0.19 s, about 15% and 0.08% compared to DPN and DPC, respectively. The time spent by DPR on each instance is within half a second. The worst-case for DPR (0.51 s) is more than eight times better than the worst-case for DPN (4.32 s). Not surprisingly, the time spent by DPR on instances with multiple defects is generally more than the one with a single defect. The time spent by DPM on each instance is less than 0.5 s, and the average time spent by DPM is 0.11 s, which is less than DPR. This can be attributed to the fewer discrete points within the MIM patterns than the raster points in these instances. However, the time complexity of the DP approach is cubic to the discrete point (Afsharian et al., 2014), therefore, the ratio of execution times is much better than the reduction of coordinate numbers.

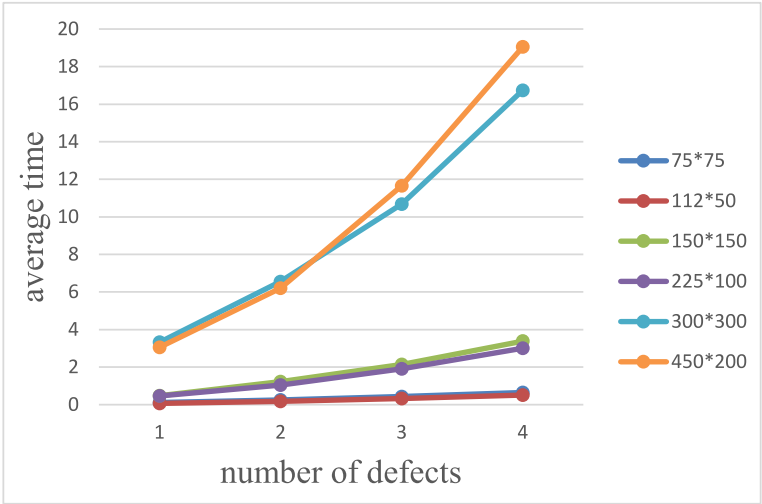


Fig. 3. The average time spent by DPR for instances of second set with a different number of defects.

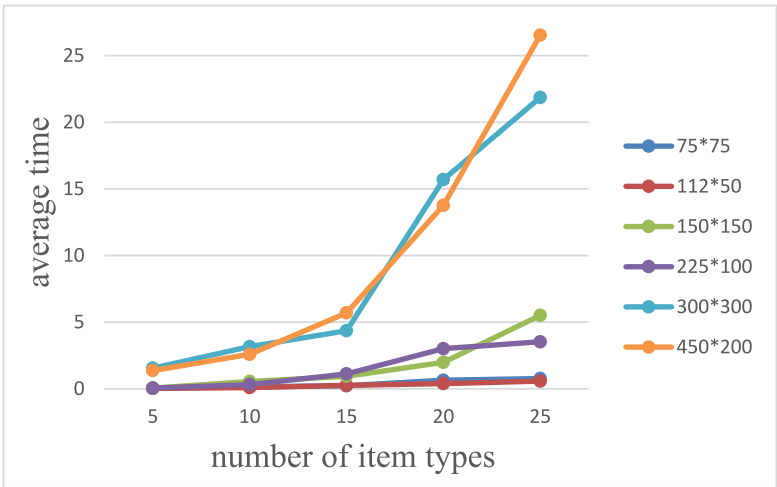


Fig. 4. The average time spent by DPR for instances of second set with a different number of item types.

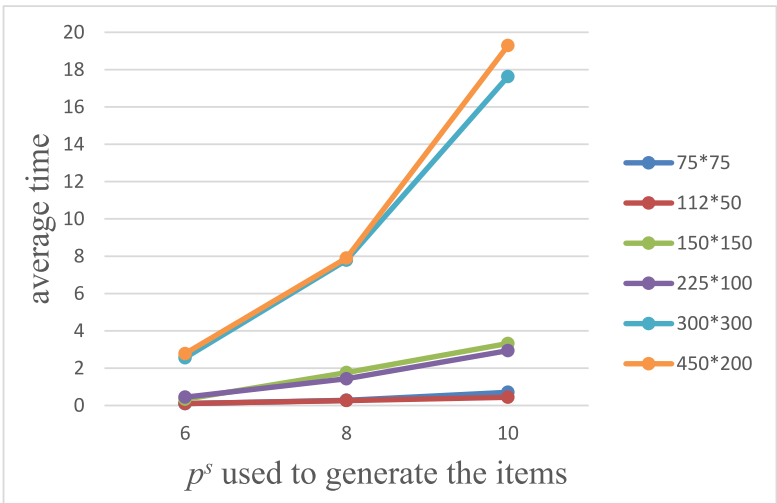
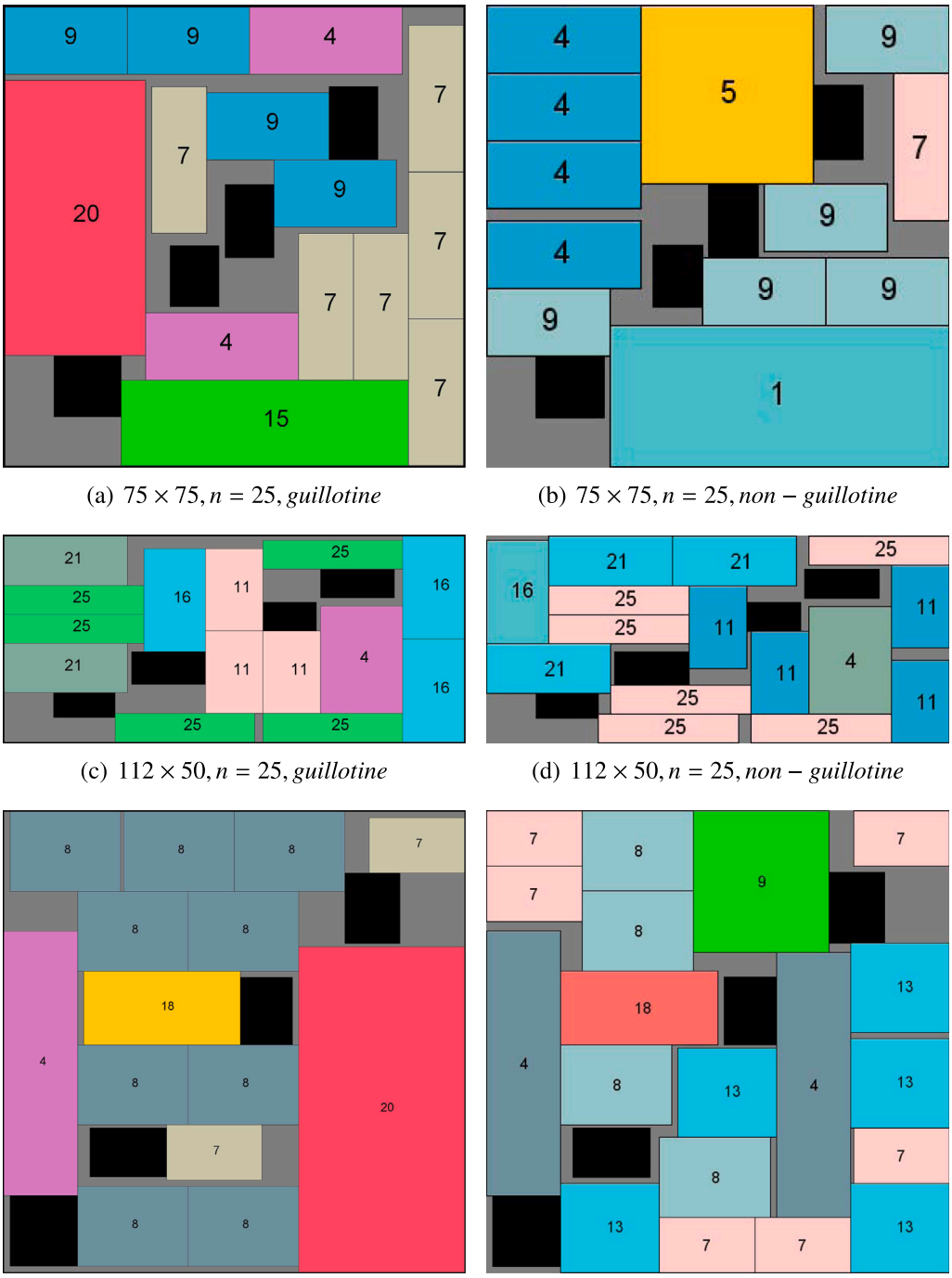


Fig. 5. The average time spent by DPR for instances of second set generated by different p^s values.



(a) $75 \times 75, n = 25, guillotine$ (b) $75 \times 75, n = 25, non - guillotine$

(c) $112 \times 50, n = 25, guillotine$ (d) $112 \times 50, n = 25, non - guillotine$

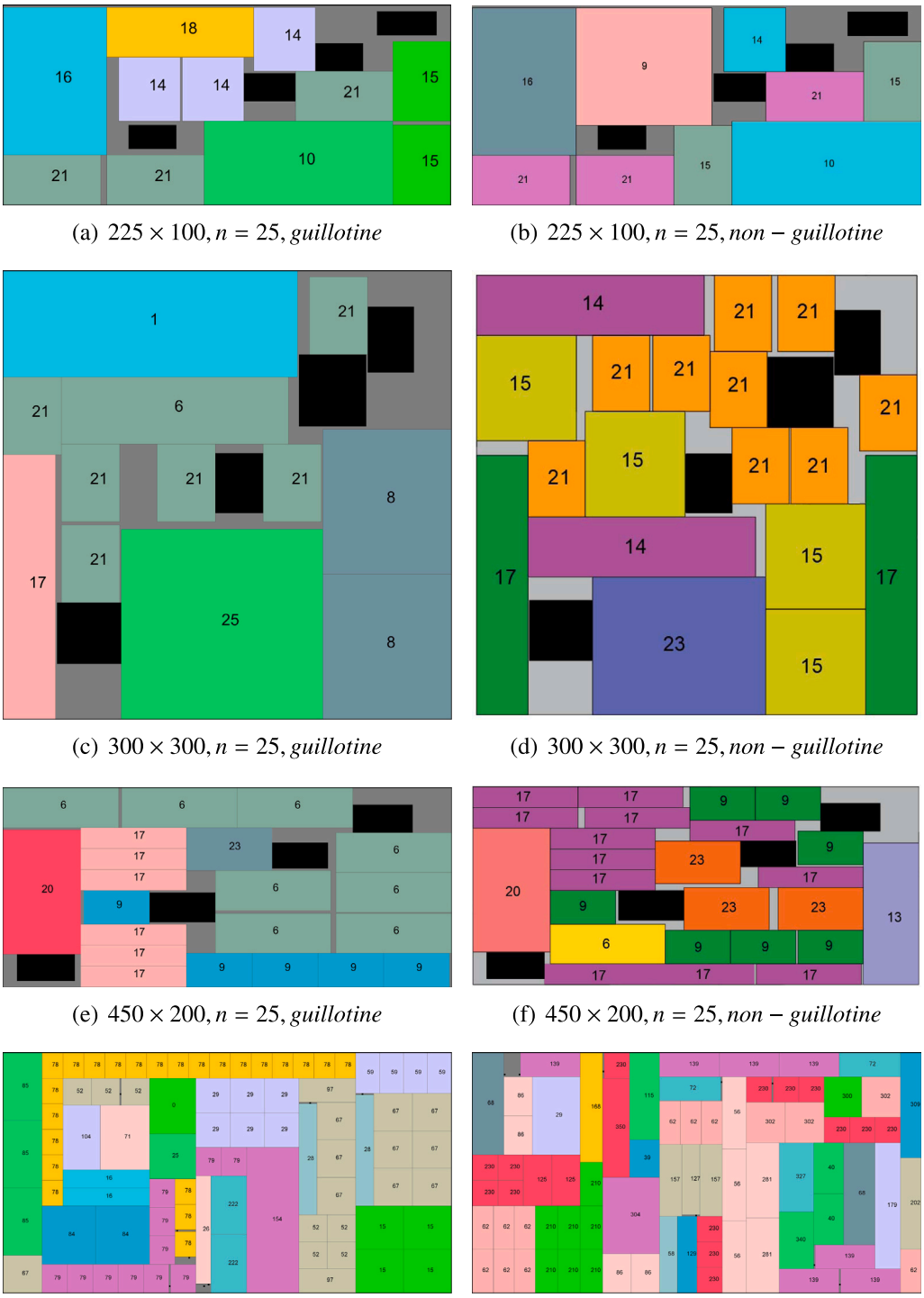
(e) $150 \times 150, n = 25, guillotine$ (f) $150 \times 150, n = 25, non - guillotine$

Fig. A.6. Layout for some selected instances using MIM patterns.

6.4.2. Results for the second set

The comparative results between DPD, IHDP, B&BC, and our approaches on the instances of category 1 in the second set are shown in Table 9, where *OFV* means the average objective function value and t (s) is the average time per instance in each class. Among DPD, IHDP, and B&BC, only IHDP found the optimal solution for all the instances of this category. While DPD found the optimal solution for the 82.22% of the instances, and B&BC found the optimal solution for the 79.89% according to its time limit. The average time used by DPN is about 1.53 s, about 3.3% to the one used by DPC (47.2 s). The average time spent by DPR is 0.32 s, about 21% and 0.68% compared to DPN and DPC, respectively. The average time required by DPR and DPM for each class is less than 3 s. The time used by DPN, DPR, and DPM is much less than the approaches DPD, IHDP, and B&BC.

Since Martin et al. (2020) (for B&BC) only reported the results of category 1 in the second set, we only compare the results of DPD, IHDP, DPC, DPN, DPR, and DPM for the instances of category 2. The results are shown in Table 10. DPD obtains none of the optimal solutions for these instances, and IHDP receives 26 out of 30 class optimal solutions. However, the average time for IHDP is 125 s, which is about twice as long as DPD. The average time spent by DPC is 1034 seconds, while DPN only requires 15.9 s which is 1.54% compared to DPC.



(a) $225 \times 100, n = 25, guillotine$

(b) $225 \times 100, n = 25, non - guillotine$

(c) $300 \times 300, n = 25, guillotine$

(d) $300 \times 300, n = 25, non - guillotine$

(e) $450 \times 200, n = 25, guillotine$

(f) $450 \times 200, n = 25, non - guillotine$

(g) $300 \times 160, n = 274, guillotine$

(h) $300 \times 160, n = 363, guillotine$

Fig. A.7. Layout for some selected instances using MIM patterns.

Moreover, the average time spent by DPR is 1.7 s, about 10.7% and 0.16% compared to DPN and DPC, respectively. The maximum average time DPR uses is about 10 s for class 15. The average time spent by DPM is 2.0 s, a bit more than DPR.

For the instances of category 3, since Afsharian et al. (2014) (for DPD) and Yin et al. (2019) (for IHDP) have not reported results for these instances, and DPC requires too much time to solve them (more than 10 h for several tried instances), we only compare the results of DPN, DPR, and DPM. The results are shown in Table 11. The average time spent by DPN is 235.6 s, while DPR only requires 9.7 s which is 4.12% compared to DPN. The maximum average time DPR uses is about 53 s for class 30. The average time spent by DPM is 13.0 s, worse than DPR.

6.4.3. Results for the third set

The comparative results between DPN, DPR, and DPM on the instances in the third set are shown in Tables 12, 13, and 14, where each table is for one of the three subsets in the third set. We can see that for the instances without defects ($m = 0$), all these approaches find

the optimal solution within a short time. The instance requiring the maximum time is B12 with 439 items and six defects. The time required by DPN, DPR, and DPM is 5167, 3232, and 3129 seconds, respectively. On average, the time required by DPM is about half with respect to DPN and less than DPR.

6.4.4. Effect of different factors

To investigate the effect of different factors on the time used by DPR, we classified the instances of the second set into six subsets (each with 5 groups, that is, 15 classes) by the size of the input sheet. Then, we divided the instances in each subset further into different partitions based on different factors and averaged the time used by each partition.

Firstly, for four partitions based on m, we investigate the effect of the defects number. The result is shown in Fig. 3, where each line represents a subset and the x-axis represents the number of defects. The average time per instance in each partition is shown on the y-axis. This figure indicates that the time required by DPR increases almost linearly with the number of defects. This is because the number of raster points is closely related to the number of defects.

For five partitions based on n, the effect of the number of item types is investigated afterward. The result is plotted in Fig. 4, where the x-axis is for the number of the item types. Similar to the number of defects, the time required increases with the number of item types, but the trend is not strictly linear.

Finally, for three partitions based on p^s, we investigate the effect of the parameter p^s used to control the size of item types. Given that the width and height of the items are generated uniformly in the interval $[W_0/p^s, 3W_0/4]$ and $[H_0/p^s, 3H_0/4]$, respectively, the larger p^s, the larger the length of the interval. In other words, the size of the item types will be more diverse for larger p^s. Fig. 5 plots the effect of the parameter p^s on the average time used. The results indicate that the larger p^s, the more time DPR requires. This is because the number of raster points increases with the diversity of the items size.

7. Conclusion

This paper studies the unconstrained two-dimensional cutting problem with defects. Two versions, guillotine, and non-guillotine, are studied. The normal patterns, raster points, and MIM patterns have been widely used in cutting literature to reduce vertical and horizontal cuts. This paper shows that these concepts can be extended to solve the cutting problem with defects without losing optimality.

For the non-guillotine version, an integer model was proposed. The results show that the proposed model can solve almost all the instances in the literature. For the guillotine-cut version, four different approaches, DPC (DP considering all integer points), DPN (DP based on the normal patterns), DPR (DP based on raster points), and DPM (DP based on MIM patterns), were implemented. The standard benchmark experiments show that the DPN can reduce the time by more than 80% compared to DPC, whereas DPR and DPM can reduce it by more than 99%.

Further work is to extend the proposed approach to solve other cutting problems with defects, such as the two-dimensional strip packing with defects and the two-dimensional bin packing with defects. It can also be generalized to address higher-dimensional problems.

Declaration of competing interest

We declare that we have no financial and personal relationships with other people or organizations that can inappropriately influence our work, there is no professional or other personal interest of any nature or kind in any product, service and/or company that could be construed as influencing the review of, the manuscript entitled.

Data availability

Data will be made available on request

Acknowledgments

The authors thank the anonymous reviewers for their helpful comments. This paper is supported by the Science Fund for Distinguished Young Scholars of Guangdong Province (No. 2022B1515020076), the Natural Science Foundation of China (Nos. 72271062), the Province Natural Science Fund of Guangdong (No. 2021A1515012459), and the Funding by Science and Technology Projects in Guangzhou (No. 2023A04J0407).

Appendix. Packing results for some selected instances

Figs. A.6 and A.7 give the layout gotten by our approach for some selected instances.

References

Afsharian, M., Niknejad, A., Wäscher, G., 2014. A heuristic,dynamic programming-based approach for a two-dimensional cutting problem with defects. OR Spectr. 36, 971–999.

Beasley, J., 1985a. Algorithms for unconstrained two-dimensional guillotine cutting. J. Oper. Res. Soc. 36 (4), 297–306.

Beasley, J.E., 1985b. An exact two-dimensional non-guillotine cutting tree search procedure. Oper. Res. 33 (1), 49–64.

Birgin, E.G., Lobato, R.D., Morabito, R., 2012. Generating unconstrained two-dimensional non-guillotine cutting patterns by a Recursive Partitioning Algorithm. J. Oper. Res. Soc. 63 (2), 183–200.

Boschetti, M.A., Maniezzo, V., Strappaveccia, F., 2016. Using GPU computing for solving the two-dimensional guillotine cutting problem. INFORMS J. Comput. 28 (December), 540–552.

Carnieri, C., Mendoza, G.A., Gavinho, L.G., 1994. Solution procedures for cutting lumber into furniture parts. European J. Oper. Res. 73 (3), 495–501.

Carnieri, C., Mendoza, G.A., Luppold, W.G., 1993. Optimal cutting of dimension parts from lumber with a defect: a heuristic solution procedure. For. Prod. J. 43 (9), 66–72.

Christofides, N., Whitlock, C., 1977. An algorithm for two-dimensional cutting problems. Oper. Res. 25 (1), 30–44.

Côté, J.-F., Iori, M., 2018. The Meet-in-the-Middle principle for cutting and packing problems. INFORMS J. Comput. 30 (June), 646–661.

de Almeida Cunha, J.G., de Lima, V.L., de Queiroz, T.A., 2020. Grids for cutting and packing problems : a study in the 2D knapsack problem. 4OR 18 (3), 293–339.

Durak, B., Tuzun Aksu, D., 2017. Dynamic programming and mixed integer programming based algorithms for the online glass cutting problem with defects and production targets. Int. J. Prod. Res. 75 (24), 7398–7411.

Fayard, D., Hifi, M., Zissimopoulos, V., 1998. An efficient approach for large-scale two- dimensional guillotine cutting stock problems. J. Oper. Res. Soc. 49 (12), 1270–1277.

Fischer, A., Scheithauer, G., 2015. Cutting and packing problems with placement constraints. In: Fasano, G., Pintér, J.D. (Eds.), Optim. Packings with Appl.. Springer, pp. 119–156.

Gilmore, P.C., Gomory, R.E., 1961. A linear programming approach to the cutting-stock problem. Oper. Res. 9 (6), 849–859.

Gilmore, P.C., Gomory, R.E., 1966. The theory and computation of knapsack functions. Oper. Res. 14 (6), 1045–1074.

Gonçalves, J.F., Wäscher, G., 2020. A MIP model and a biased random-key genetic algorithm based approach for a two-dimensional cutting problem with defects. European J. Oper. Res. 286, 867–882.

Herz, J.C., 1972. Recursive computational procedure for two-dimensional stock sutting. IBM J. Res. Dev. 16 (5), 462–469.

Hifi, M., 1997. The DH / KD algorithm : a hybrid approach for unconstrained two-dimensional cutting problems. European J. Oper. Res. 97 (1), 41–52.

Hifi, M., Zissimopoulos, V., 1996. A recursive exact algorithm for weighted two-dimensional cutting. European J. Oper. Res. 91 (3), 553–564.

Iori, M., de Lima, V.L., Martello, S., Miyazawa, F.K., Monaci, M., 2021. Exact solution techniques for two-dimensional cutting and packing. European J. Oper. Res. 289 (2), 399–415.

Kang, M.-K., Yoon, K., 2011. An improved best-first branch-and-bound algorithm for unconstrained two-dimensional cutting problems. Int. J. Prod. Res. 49 (15), 4437–4455.

Libralesso, L., Fontan, F., 2021. An anytime tree search algorithm for the 2018 ROADEF/EURO challenge glass cutting problem. European J. Oper. Res. 291 (3), 883–893.

Lins, L., Lins, S., Morabito, R., 2003. An L-approach for packing (l, w)-rectangles into rectangular and L-shaped pieces. J. Oper. Res. Soc. 54 (7), 777–789.

Luo, Q., Rao, Y., Guo, X., Du, B., 2022. A biased genetic algorithm hybridized with VNS for the two-dimensional knapsack packing problem with defects. Appl. Soft Comput. 118, 108479.

Martin, M., Hokama, P.H.D.B., Morabito, R., Munari, P., 2020. The constrained two-dimensional guillotine cutting problem with defects : an ILP formulation , a Benders decomposition and a CP-based algorithm. Int. J. Prod. Res. 58 (9), 2712–2729.

Martin, M., Morabito, R., Munari, P., 2022. Two-stage and one-group two-dimensional guillotine cutting problems with defects: a CP-based algorithm and ILP formulations. Int. J. Prod. Res. 60 (6), 1854–1873.

Masone, A., Russo, M., Sterle, C., 2022. Improved dynamic programming algorithms for unconstrained two-dimensional guillotine cutting. Available at SSRN: http://dx.doi.org/10.2139/ssrn.4239711.

Morabito, R., Morales, S., 1998. A simple and effective recursive procedure for the manufacturer's pallet loading problem. J. Oper. Res. Soc. 49 (8), 819–828.

Neidlein, V., Scholz, A., Wäscher, G., 2016. SLOPPGEN: a problem generator for the two-dimensional rectangular single large object placement problem with defects. Int. Trans. Oper. Res. 23, 173–186.

Neidlein, V., Vianna, A., Arenales, M., Wäscher, G., 2009. Two-dimensional guillotineable-layout cutting problems with a single defect - an AND/OR-Graph approach. In: Fleischmann, B., Borgwardt, K., Klein, R., Tuma, A. (Eds.), Operations Research Proceedings 2008. Springer Berlin Heidelberg, pp. 85–90.

Parreño, F., Alonso, M.T., Alvarez-Valdes, R., 2020. Solving a large cutting problem in the glass manufacturing industry. European J. Oper. Res. 287 (1), 378–388.

Parreño, F., Alvarez-Valdes, R., 2021. Mathematical models for a cutting problem in the glass manufacturing industry. Omega 103, 102432.

de Queiroz, T.A., Miyazawa, F.K., Wakabayashi, Y., 2014. On the *L*-approach for generating unconstrained two-dimensional non-guillotine cutting patterns. 4OR 13 (2), 199–219.

Rao, Y., Luo, Q., 2022. Solutions for new variants of packing problem. In: Intell. Algorithms Pack. Cut. Probl.. Springer Nature Singapore, Singapore, pp. 133–197.

Russo, M., Boccia, M., Sforza, A., Sterle, C., 2020. Constrained two-dimensional guillotine cutting problem : upper-bound review and categorization. Int. Trans. Oper. Res. 27, 794–834.

Russo, M., Sforza, A., Sterle, C., 2013. An improvement of the knapsack function based algorithm of Gilmore and Gomory for the unconstrained two dimensional guillotine cutting problem. Int. J. Prod. Econ. 145, 451–462.

Russo, M., Sforza, A., Sterle, C., 2014. An exact dynamic programming algorithm for large-scale unconstrained two-dimensional guillotine cutting problems. Comput. Oper. Res. 50, 97–114.

Scheithauer, G., 2018. Orthogonal Packing Feasibility, Two-Dimensional Knapsack Problems. In: Introd. To Cut. Pack. Optim. Probl. Model. Approaches, Solut. Methods. Springer International Publishing, Cham, pp. 123–156.

Scheithauer, G., Terno, J., 1996. The G4-heuristic for the pallet loading problem. J. Oper. Res. Soc. 47 (4), 511–522.

Terno, J., Lindemann, R., Scheithauer, G., 1987. Zuschnittprobleme und ihre praktische Lösung. Technical Report, Verlag Harry Deutsch, Thun und Frankfurt, Germany.

Tuzun Aksu, D., Durak, B.r., 2016. A dynamic programming algorithm for the online cutting problem with defects and quality grades. IFAC-PapersOnLine 49 (12), 17–22.

Vianna, A.C.G., Arenales, N.M., 2006. Problema de corte de placas defeituosas. Pesqui. Operacional 26, 185–202.

Wäscher, G., Haußner, H., Schumann, H., 2007. An improved typology of cutting and packing problems. European J. Oper. Res. 183 (3), 1109–1130.

Wei, L., Lim, A., 2015. A bidirectional building approach for the 2D constrained guillotine knapsack packing problem. European J. Oper. Res. 242 (1), 63–71.

Wei, L., Oon, W.-C.C., Zhu, W., Lim, A., 2013. A goal-driven approach to the 2D bin packing and variable-sized bin packing problems. European J. Oper. Res. 224 (1), 110–121.

Wei, L., Tian, T., Zhu, W., Lim, A., 2014. A block-based layer building approach for the 2D guillotine strip packing problem. European J. Oper. Res. 239 (1), 58–69.

Wei, L., Wang, Y., Cheng, H., Huang, J., 2019. An open space based heuristic for the 2D strip packing problem with unloading constraints. Appl. Math. Model. 70, 67–81.

Yin, A., Chen, C., Hu, D., Huang, J., Yang, F., 2019. An improved heuristic-dynamic programming algorithm for rectangular cutting problem. In: Int. Symp. Parallel Archit. Algorithms Program.. Springer, pp. 221–233.

Young-Gun, G., Kang, M.-K., 2002. A new upper bound for unconstrained two-dimensional cutting and packing. J. Oper. Res. Soc. 53 (5), 587–591.

Young-Gun, G., Seong, Y.-J., Kang, M.-K., 2003. A best-first branch and bound algorithm for unconstrained two-dimensional cutting problems. Oper. Res. Lett. 31 (4), 301–307.

Zhang, H., Yao, S., Liu, Q., Wei, L., Lin, L., Leng, J., 2023. An exact approach for the constrained two- dimensional guillotine cutting problem with defects. Int. J. Prod. Res. 61 (9), 2985–3002.